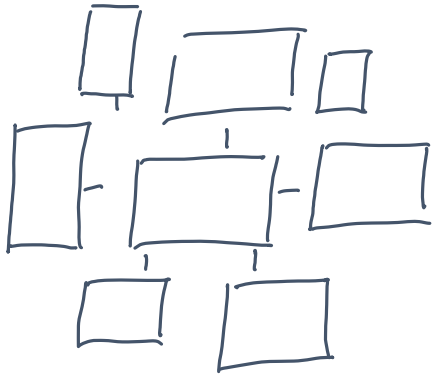




FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Introduction to SystemVerilog HDL design

Today's Agenda

Intended topics for today's session

- What is an HDL and how to learn it?
- Learning by example – A first taste: A series of basic combinational and sequential SystemVerilog designs

Appendix

- History of SystemVerilog



FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

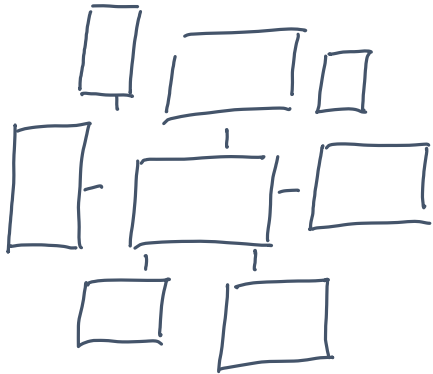
fbeit

Introduction to SystemVerilog HDL design

Objectives

By the end of this lecture you will be able to ...

- understand the basic structure of a SystemVerilog module
- design simple combinational circuits in SystemVerilog



FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

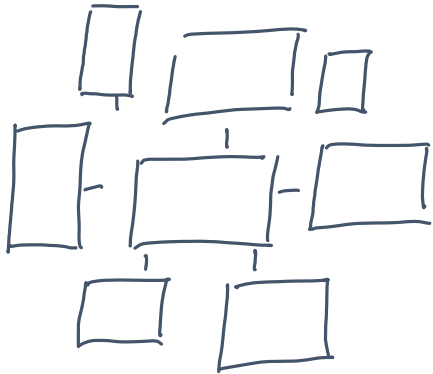
fbeit

Introduction to SystemVerilog HDL design

Recommended Readings

Textbooks, Application Notes, White Papers ...

- Sutherland, S., “RTL Modeling with SystemVerilog for Simulation and Synthesis: Using SystemVerilog for ASIC and FPGA Design”, CreateSpace Independent Publishing Platform, 2017
- Spear, C., “SystemVerilog for Verification: A Guide to Learning the Testbench Language Features”, Springer, 3rd edition, 2012.



Introduction to SystemVerilog HDL – Part #1

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

What is an HDL?

- A Language to describe, simulate, and create hardware (popular examples are VHDL, Verilog or SystemVerilog).
- HDLs are an integral part of modern electronic design automation (EDA) systems.
- Despite similar syntax, an HDL cannot be used like typical programming languages
- Express the dimensions of timing and concurrency.
- At Register Transfer Level (RTL), an HDL design describes a hardware structure, not an algorithm.
- At behavioral level, HDL models describe only the behavior of the design with no implied structure.
- Something that you should keep in mind: “If you can’t draw it, don’t try to code it!”

How to learn an HDL ...

- Learning by doing.
- Learning from mistakes.
- Try to understand what and why something went wrong ... Otherwise nothing has been learned.
- Start designing simple designs, slowly add complexity.
- Start your design on paper ...

SystemVerilog at a glance ...

- SystemVerilog represents a unified hardware design, specification and verification language.
- It provides support for all Verilog constructs (Verilog-2005). In addition, it combines synthesizable constructs from Accelera's language Superlog and Verification constructs from Synopsys OpenVera.
- In general, the feature-set of SystemVerilog (IEEE standard 1800-2012) can be divided into two distinct sections:
 1. SystemVerilog for RTL design is an extension of Verilog-2005; all features of that language are available in SystemVerilog.
 2. SystemVerilog for verification uses extensive object-oriented programming techniques and is more closely related to Java than Verilog.

SystemVerilog at a glance ...

- It is important to understand that SystemVerilog is both a 'synthesis' and 'simulation/verification' language.
- A certain subset of the language is used for synthesizing a hardware description into dedicated set of logic gates and flip-flops.
- Another subset of the language provides features for simulation and verification purposes. These constructs can not be translated into equivalent hardware structures ...

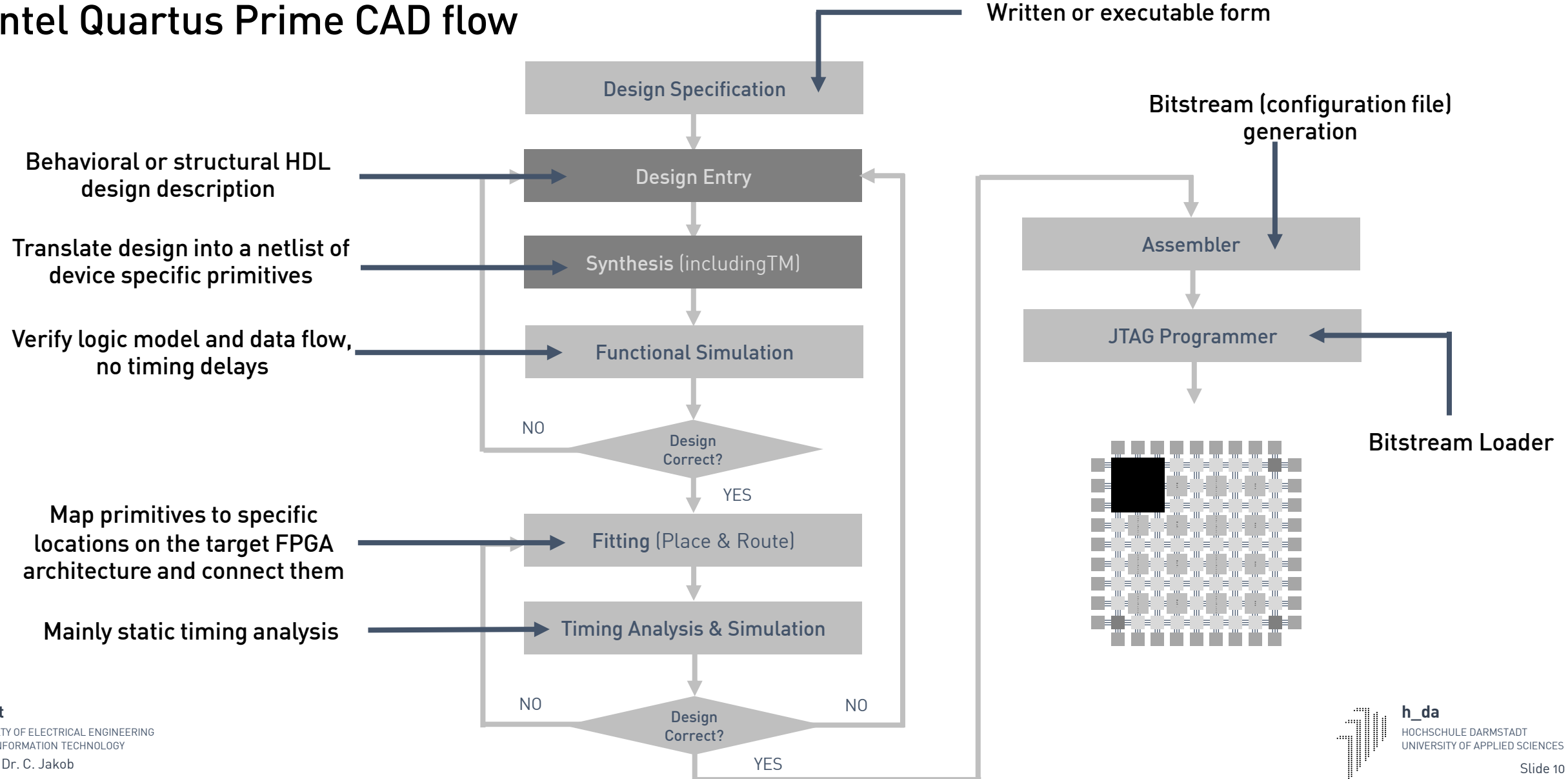
SystemVerilog at a glance ...

- Despite the fact that SystemVerilog syntactically looks like 'C', it is no software programming language.
- This leads to the point that certain construct can be easily misinterpreted.
- It is a good strategy to think of the hardware synthesized that each line of SystemVerilog code will produce.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

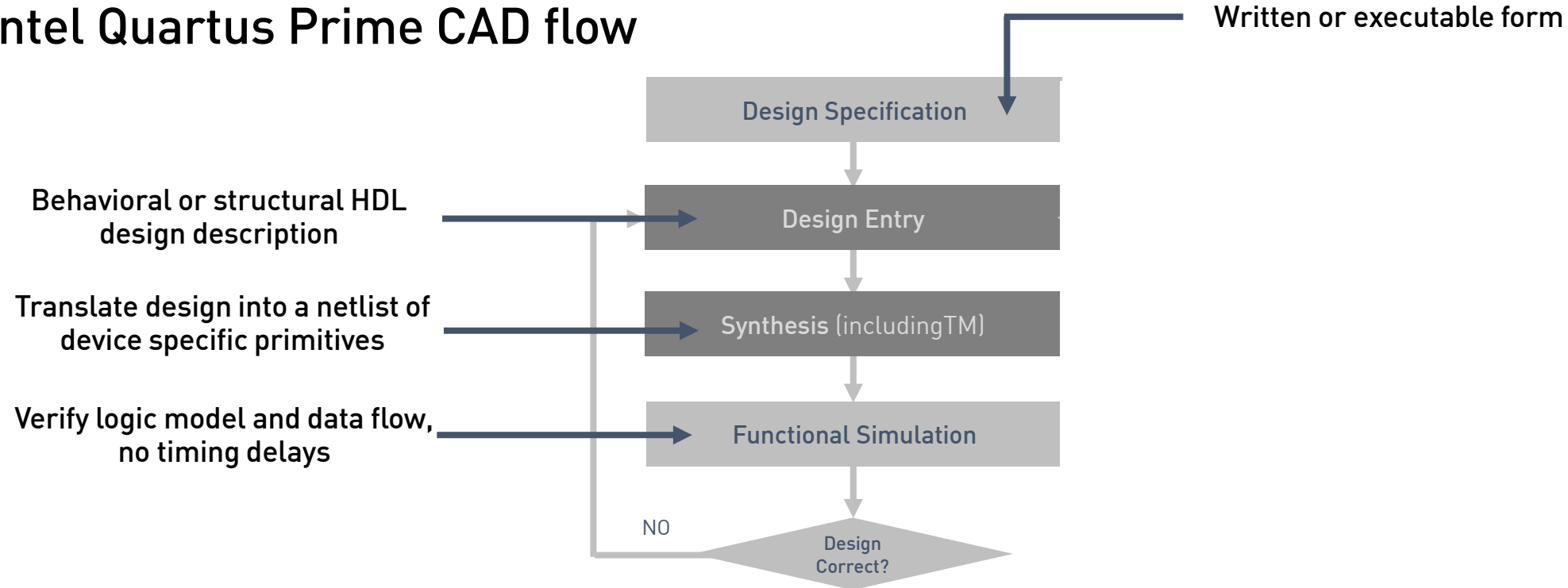
Intel Quartus Prime CAD flow



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Intel Quartus Prime CAD flow



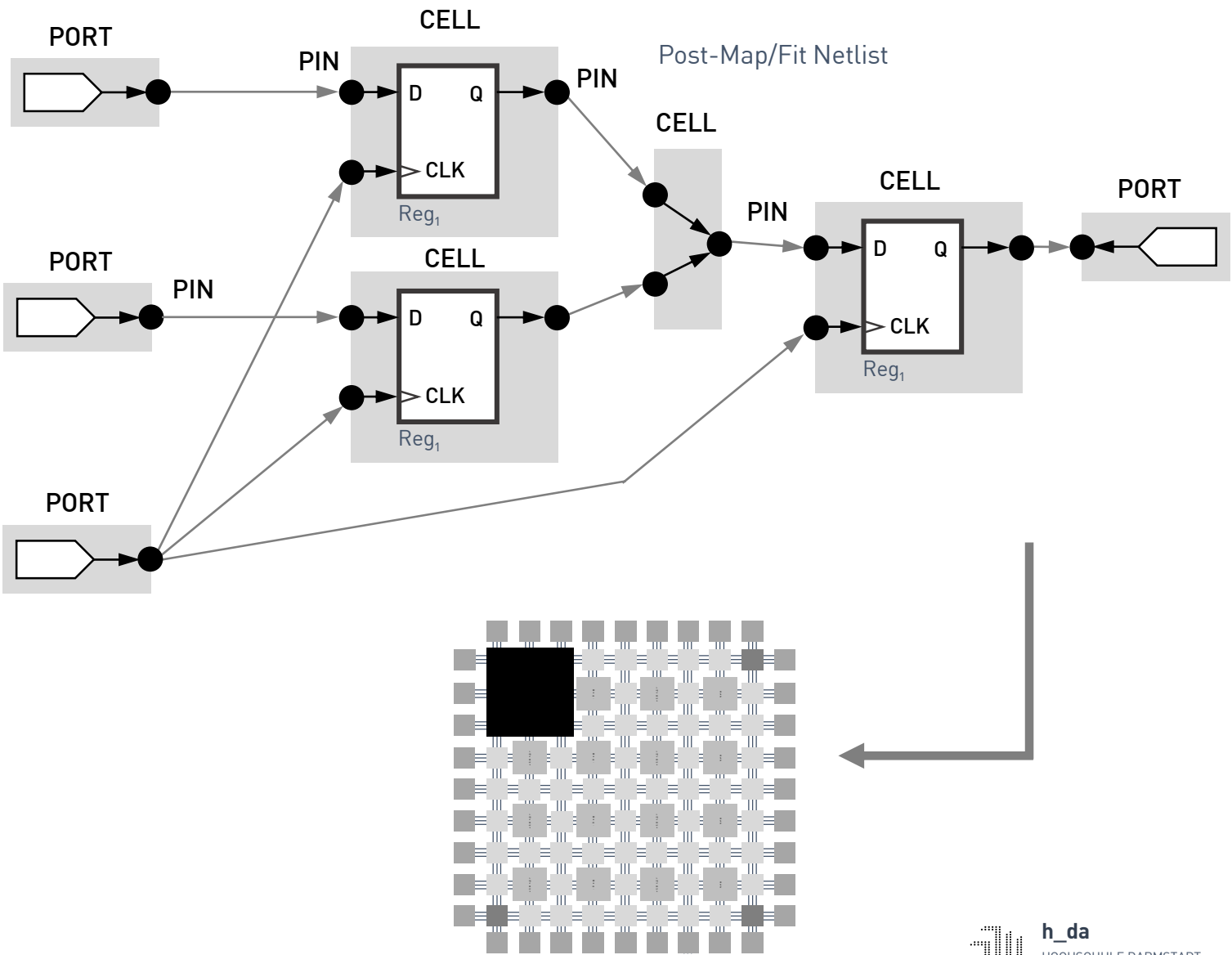
Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

What is an HDL?

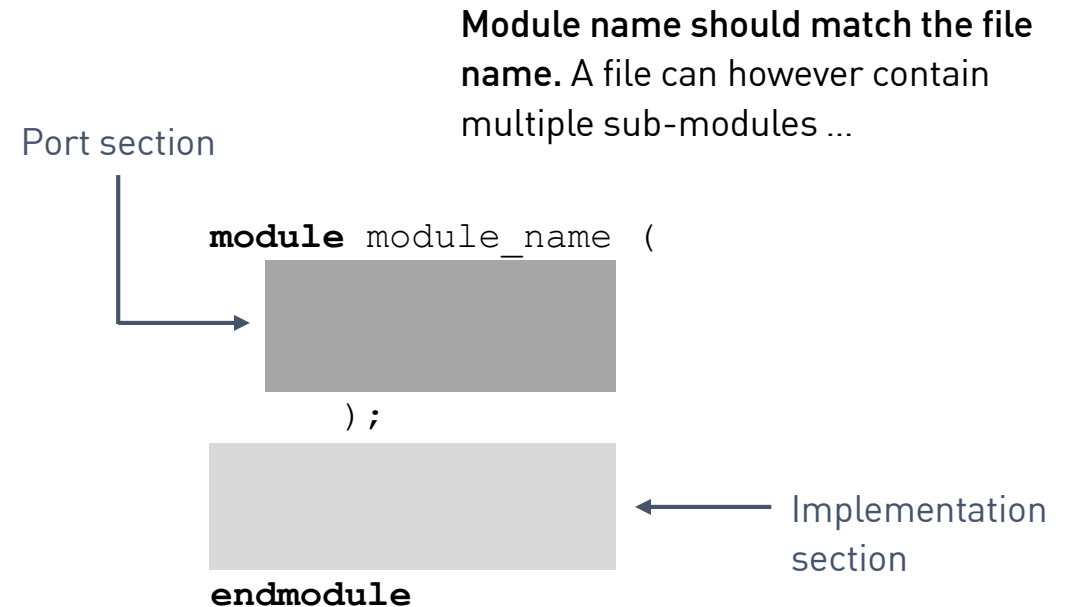
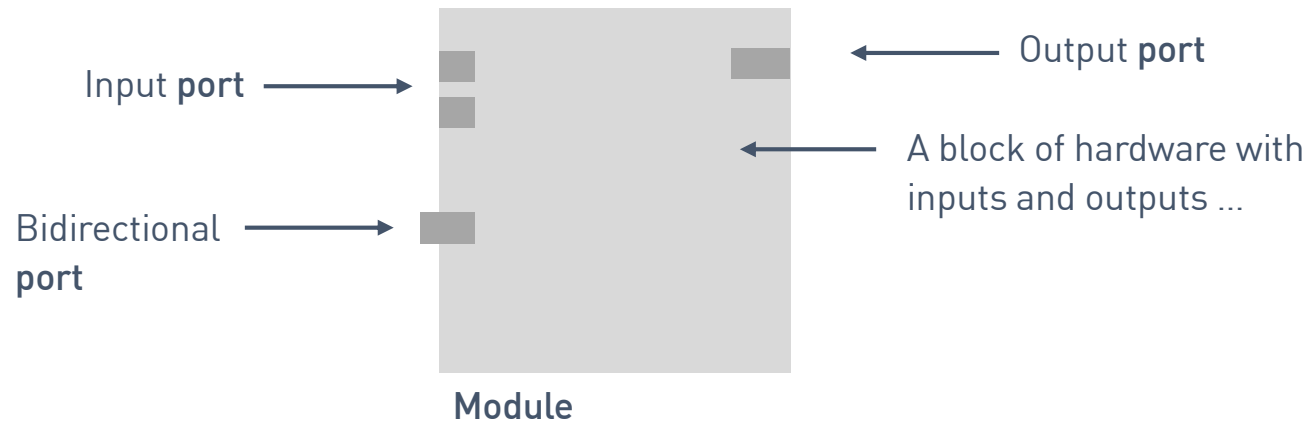
```
1. module sys_proc(  
2.     input logic [3:0] d0, d1,  
3.     ...  
4. );  
5.   
6.     always_ff@(posedge clk)  
7.         if(reset_n == 1'b1)  
8.             q <= 1'b0;  
9.         ...  
10.   
11. endmodule
```

[SystemVerilog] Source Code Excerpt: sys_proc.sv



The Module – The basic building block in SystemVerilog

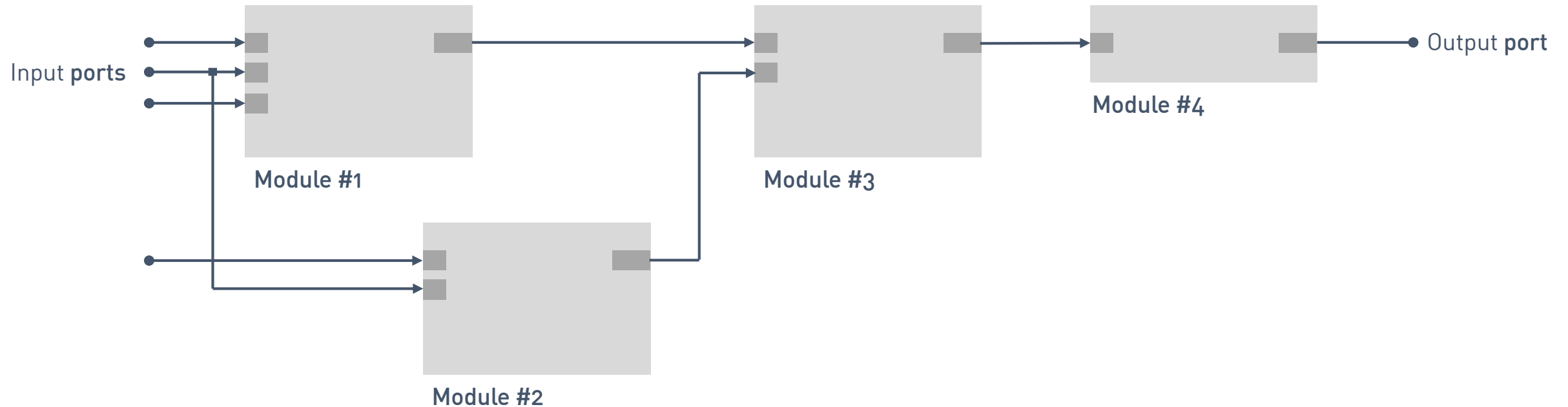
- The module is the basic unit of hierarchy in SystemVerilog.
- Modules are used to provide the **coarse-grained** structure of a design.



- The module could implement a simple AND gate, a multiplexer or even something complex such as CPU. In addition, a module can be a single element or collection of lower level modules.
- Ports are used to interface the module with the outside. They are either inputs, outputs or bidirectional (tri-state logic).
- In conclusion, modules describe the boundaries of a design unit [module, endmodule], its inputs and outputs [ports] as well how it works [behavioral or RTL code].
- SystemVerilog is able to model a design at different levels of abstraction. A high level express little detail, low levels express much. Boundaries between levels are often not well defined.

The Module – The basic building block in SystemVerilog

- Verilog designs consist of multiple interconnected modules.



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

The Module – The basic building block in SystemVerilog

Module name, must be identical
with the file name

```
1. module [module_name] (  
2.     input  [net_type] [range] port_identifier,  
3.     inout  [net_type] [range] port_identifier,  
4.     output [net_type] [range] port_identifier  
5. );  
6.   
7.   
8. endmodule
```

1/1

Port or IO
section

Implementation
section

[SystemVerilog] Source Code Snippet

Notes

- Port type **input**: The module receives data from the outside using input ports.
- Port type **output**: The module sends data to the outside using output ports.
- Port type **inout**: The module can either send or receive data through inout ports.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

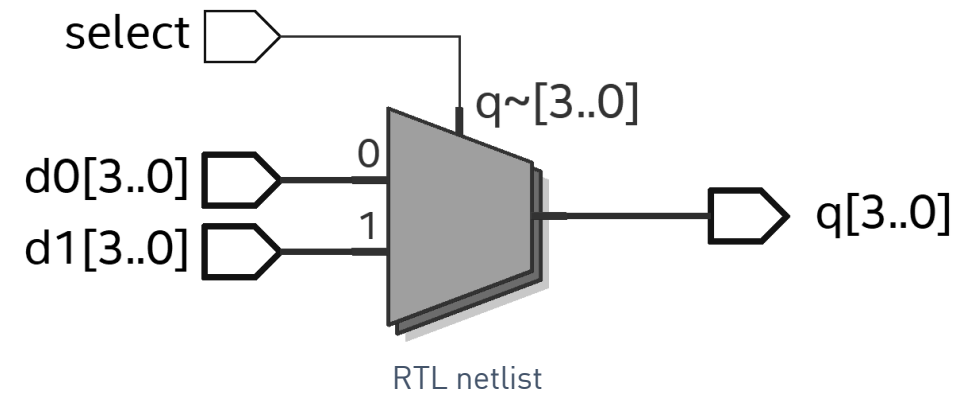
The Module – The basic building block in SystemVerilog

```
1. module mux_2_4bit_comb(  
2.     input logic [3:0] d0, d1,  
3.     input logic select,  
4.     output logic [3:0] q  
5. );  
6.     always_comb  
7.         if(select == 1)  
8.             q = d1;  
9.         else  
10.            q = d0;  
11. endmodule
```

Port or IO
section

Implementation
section

1/1



[SystemVerilog] Source Code: mux_2_4bit_comb.sv

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

The Module – The basic building block in SystemVerilog

Module name, must be identical with the file name

```
1. // ...
2. module full_adder(
3.     input logic d0, d1, cin, output logic q,
4.     output logic cout
5. );
6.
7. assign q = d0 ^ d1 ^ cin;
8. assign cout = (d0 & d1) | (d0 & cin) | (d1 & cin);
9.
10. endmodule
```

net typ: single bit 1/1

port name

continuous assignment. another way for describing combinational logic

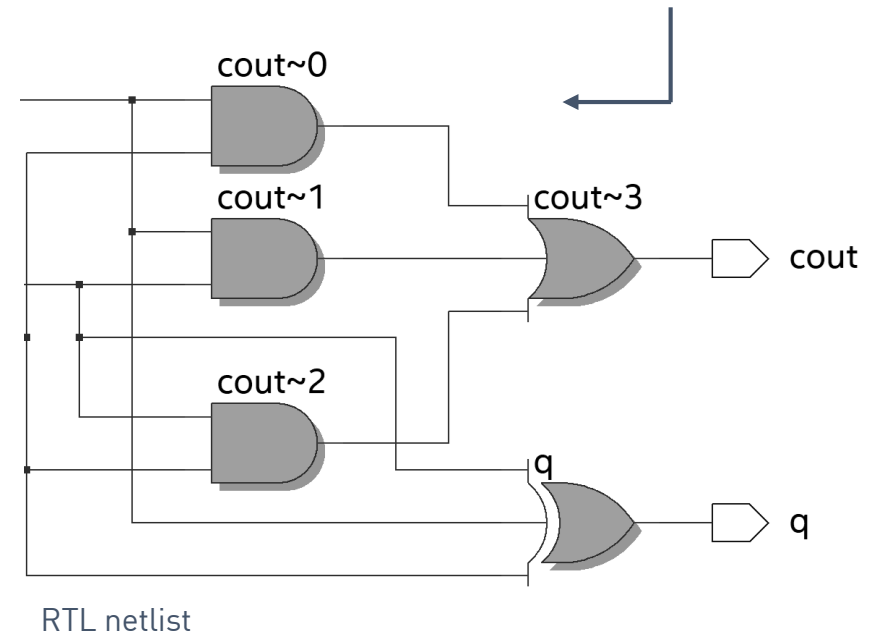
logical expression

[SystemVerilog] Source Code: full_adder.sv

Notes

- SystemVerilog is case sensitive: cin and Cin are not the same
- No names start with numbers

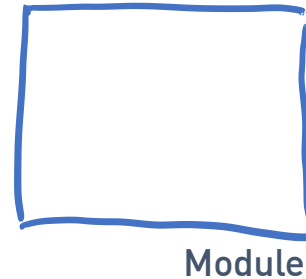
This is a **graphical representation** of the design netlist after Analysis & Elaboration and netlist extraction, but before Quartus Prime synthesis and fitting optimizations. **This RTL netlist is not the final structure of the design**, because not all optimizations are included; instead it is the closest possible view to the original RTL design



Structural and behavioral HDL descriptions

```
module full_adder(  
    input logic d0, d1, cin,  
    output logic q, cout  
);  
  
    logic tmp_sum, tmp_cout_0, tmp_cout_1;  
  
    half_adder inst_0( .d0(d0), d1(d1), .q(tmp_sum),  
                      .cout(tmp_cout_0));  
    half_adder inst_1( .d0(cin), d1(tmp_sum), .q(q),  
                      .cout(tmp_cout_1));  
  
    assign cout = tmp_cout_0 | tmp_cout_1;  
  
endmodule
```

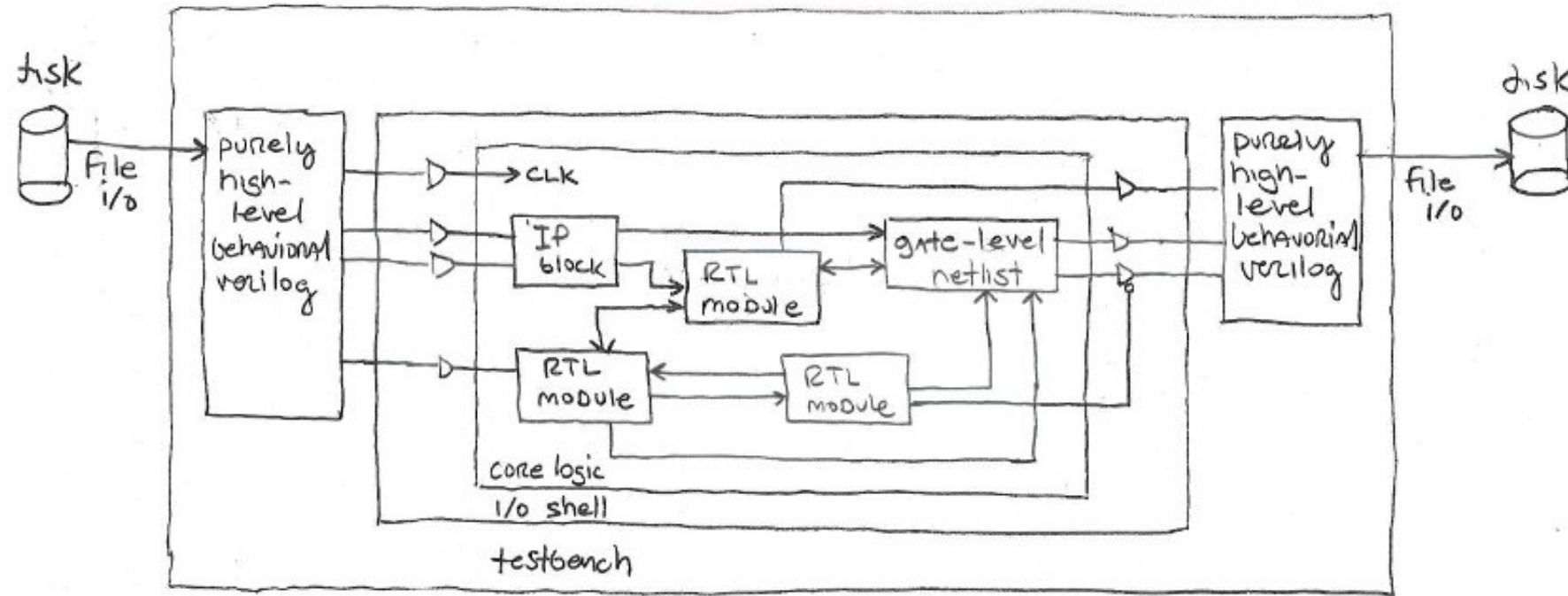
Structural description, describes
the interconnect and usage of
lower-level components



```
module add_sub_4bit(  
    input logic [3:0] d0, d1, input logic cin,  
    output logic [3:0] q, output logic cout  
);  
  
    function logic [4:0] add (input logic [3:0] a, b,  
    input logic cin );  
        logic [4:0] s; logic c; c = cin;  
        for (int i = 0; i < 4; i++) begin  
            s[i] = a[i] ^ b[i] ^ c; c = (a[i] & b[i]) | (a[i] &  
                | (c & c[i]));  
        end  
        return s;  
    endfunction  
  
    always_comb  
        if(operation) {cout, q} = adder(d0, ~d1, 1);  
        else {cout, q} = adder(d0, d1, 0);  
  
endmodule
```

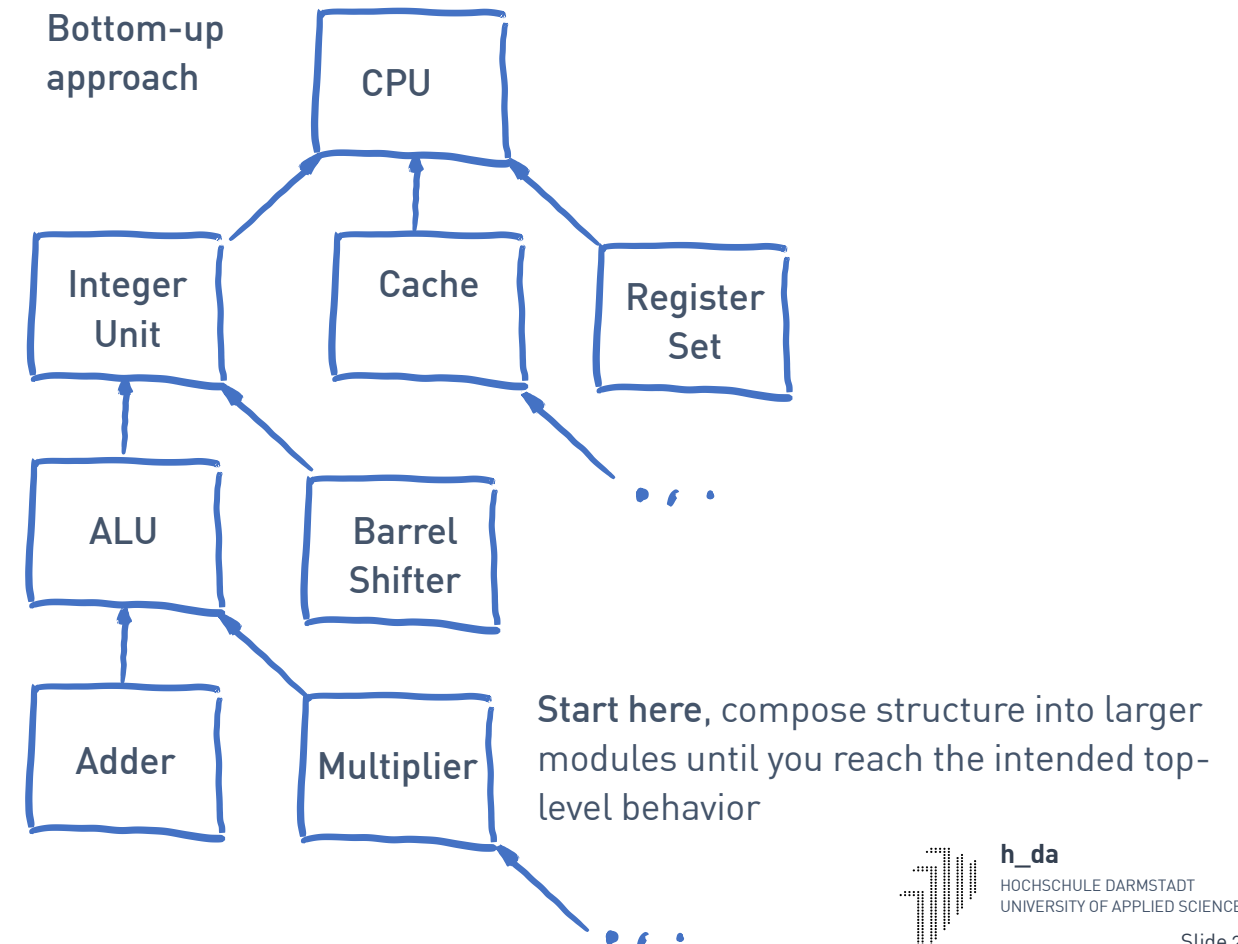
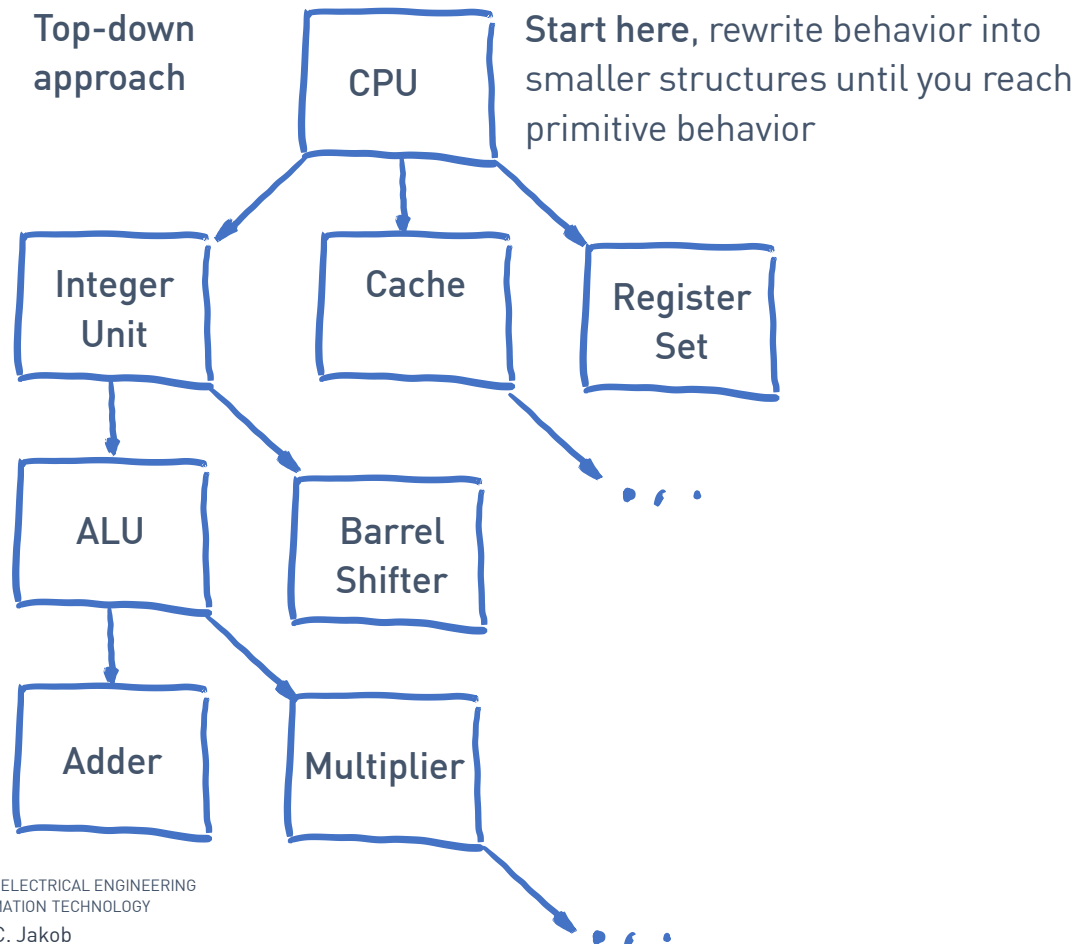
Behavioral description, describes
the algorithmic behavior of the
module rather than its structure

The Module – The basic building block in SystemVerilog



- A hierarchical design has a **top level module** and lower level ones. Lower level modules are instantiated within the higher level module. Lower level modules are connected together with wires.

Top-down and bottom-up design HDL descriptions



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Module Instantiation

As stated before, more complex designs are built by integrating multiple modules in a hierarchical manner: Modules can be **instantiated** within other modules. The ports of theses **instances** are then connected with other signals in the parent module.

```
1. // ...
2. module my_design(
3.     input logic clk, input logic reset_n,
4.     input logic data,
5.     ...
6. );
7.
8.     module_name inst_name (port_connections);
9.
10.
11. endmodule
```

Name of the top-level/parent module (points to `my_design`)

name of the instance (points to `inst_name`)

name of the module to instantiate (points to `module_name`)

port connections describe how the instantiated module is interconnected with signals in the top-level module (points to `port_connections`)

1/1

[SystemVerilog] Source Code: my_design.sv

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Module Instantiation

As stated before, more complex designs are built by integrating multiple modules in a hierarchical manner: Modules can be **instantiated** within other modules. The ports of theses **instances** are then connected with other signals in the parent module.

```
1. // ...
2. module my_design(
3.     input logic clk, input logic reset_n,
4.     input logic data,
5.     ...
6. );
7.
8.     module_name inst_name (port_connections);
9.
10.
11. endmodule
```

Annotations:

- Name of the top-level/parent module (points to `my_design`)
- name of the **instance** (points to `inst_name`)
- name of the **module to instantiate** (points to `module_name`)
- port connections describe how the instantiated module is interconnected with signals in the top-level module (points to `port_connections`)

1/1

Notes

- We can instantiate previously designed modules or pre-defined gates (and, or, xor, nand, nor, xnor)

[SystemVerilog] Source Code: my_design.sv

Introduction to SystemVerilog

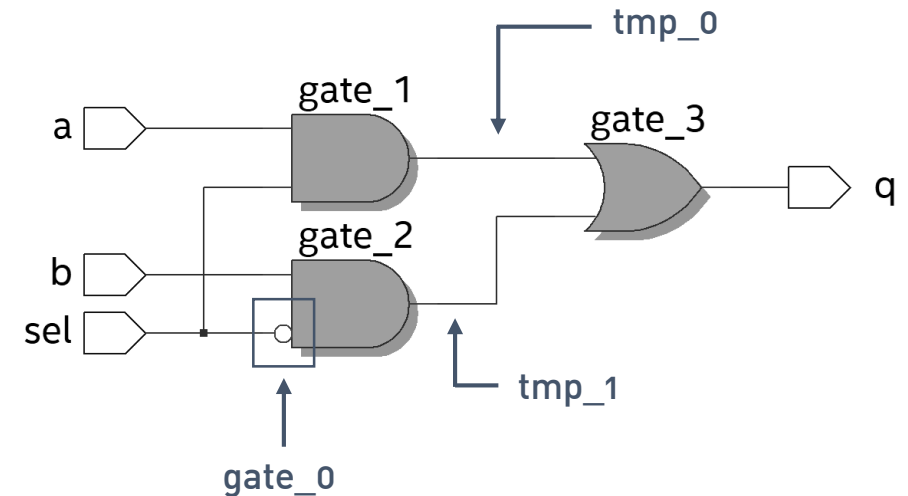
h_da – fbeit - FPGA-based SoC Design

Module Instantiation – Using pre-defined Gates

As stated before, more complex designs are built by integrating multiple modules in a hierarchical manner: Modules can be **instantiated** within other modules. The ports of these **instances** are then connected with other signals in the parent module.

```
1. // ...
2. module mux_2_1_1bit(
3.     input logic a, b, sel,
4.     output logic q
5. );
6.     intermediate wires used to
       interconnect the following logic gates.
7.     logic tmp_0, tmp_1, not_sel;
8.
9.     not gate_0(not_sel, sel);
10.    and gate_1(tmp_0, a, sel);
11.    and gate_2(tmp_1, b, not_sel);
12.    or gate_3(q, tmp_0, tmp_1);
13.
14. endmodule
    port order: output, input(s)
```

[SystemVerilog] Source Code: mux_2_1_1bit.sv



Notes

- All pre-defined gates (and, or, xor, nand, nor, xnor) have fixed port order: output, input(s).
- For self-developed modules, you can define the port order. More on that on a later point in time.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Module Instantiation – Using pre-designed Modules

```
1. module my_and_gate(  
2.     input logic a, b, output logic q  
3. );  
4.     assign q = a & b;  
5.  
6. endmodule
```

1/1

[SystemVerilog] Source Code: my_and_gate.sv

```
1. module my_top_level_design_0(  
2.     input logic d_0, d_1, output logic q_0  
3. );  
4.     my_and_gate inst_0(d_0, d_1, q_0);  
5.  
6. endmodule
```

1/1

↑
connection via port order

[SystemVerilog] Source Code: my_top_level_design_0.sv

Notes

- We can access the ports of the instantiated module **by order** or by name.

Module Instantiation

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Module Instantiation – Using pre-designed Modules

```
1. module my_and_gate(  
2.     input logic a, b, output logic q  
3. );  
4.     assign q = a & b;  
5.  
6. endmodule
```

1/1

Notes

- We can access the ports of the instantiated module by order or by name.

[SystemVerilog] Source Code: my_and_gate.sv

```
1. module my_top_level_design_1(  
2.     input logic d_0, d_1, output logic q_0  
3. );  
4.     my_and_gate inst_0(.q(q_0), .a(d_0), .b(d_1));  
5.  
6. endmodule
```

1/1

Module Instantiation

↑
connection via port name

[SystemVerilog] Source Code: my_top_level_design_1.sv

Module Instantiation

- More about interconnecting and parametrizing instantiated modules within the next lecture sessions.

Introduction to SystemVerilog

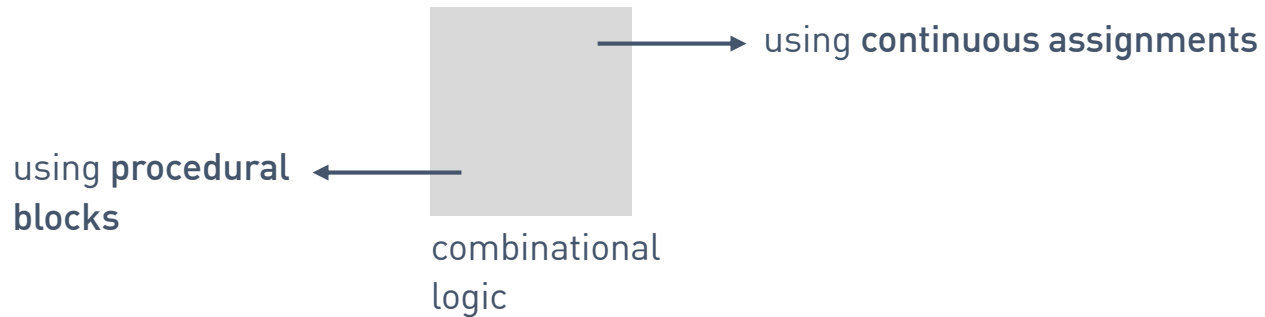
h_da – fbeit - FPGA-based SoC Design

Describing Combinational Logic using SystemVerilog

Introduction to SystemVerilog

Describing Combinational Logic using SystemVerilog

- There are two different ways to model combinational logic in SystemVerilog.

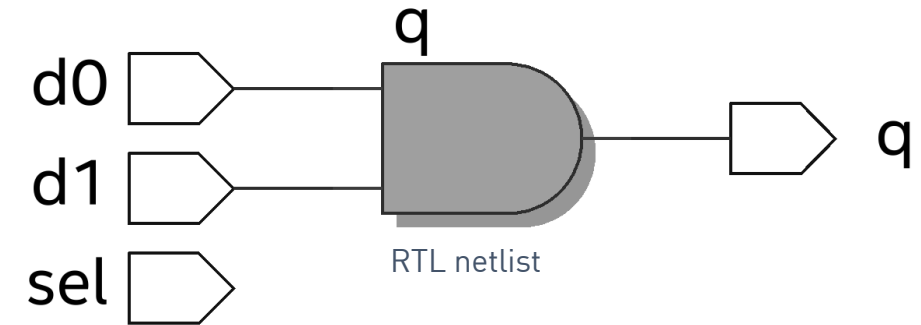


- Both are concurrent statements that may co-exist within a single module.
- In case you have a VHDL background:
 - Procedural blocks are the counterpart to processes in VHDL.
 - Continuous assignments are the SystemVerilog equivalents to signals in VHDL.

Describing Combinational Logic using SystemVerilog

```
1. module and_2_1bit(  
2.     input logic d0, d1,  
3.     output logic q  
4. );  
5.  
6.     // 2-input AND gate  
7.     assign q = d0 & d1;  
8.  
9. endmodule
```

1/1



[SystemVerilog] Source Code: and_2_1bit.sv

Notes

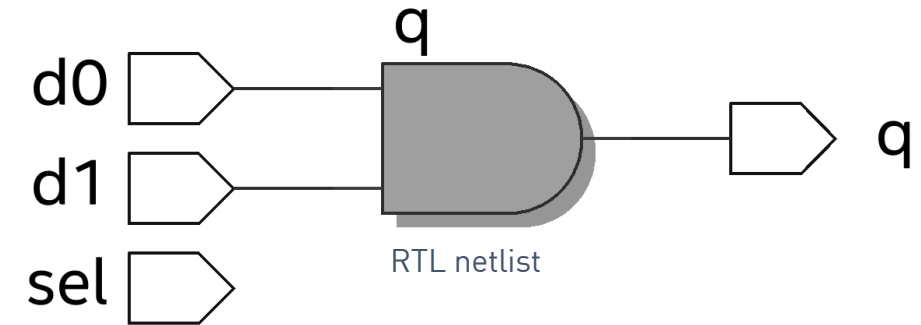
- The above example shows a so called **continuous assignments** (assign). These constructs are intended for modelling combinational logic.
- A continuous assignment drives a net similar to how a gate drives a net. The expression on the right hand side can be thought of as a combinatorial circuit that drives the net continuously.

Describing Combinational Logic using SystemVerilog

```
1. module and_2_1bit(  
2.     input logic d0, d1,  
3.     output logic q  
4. );  
5.  
6.     // 2-input AND gate  
7.     assign q = d0 & d1;  
8.  
9. endmodule
```

1/1

[SystemVerilog] Source Code: and_2_1bit.sv



Continuous assignment statements execute when a variable on the RHS changes. When the change occurs, the LHS is updated immediately. This behaviour pretty much reflects the behaviour of a real physical logic gate

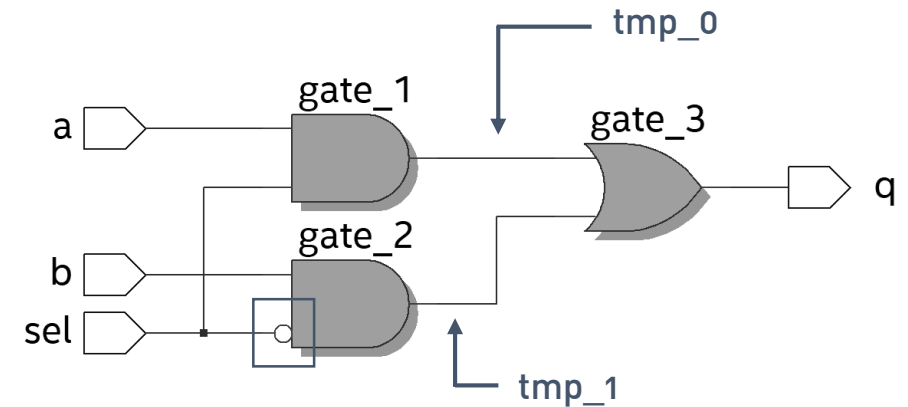
Describing Combinational Logic using SystemVerilog

```
1. module mux_2_1_1bit(  
2.     input logic a, b, sel,  
3.     output logic q  
4. );  
5.  
6.     logic tmp_0, tmp_1, not_sel;  
7.  
8.     assign tmp_0 = a & sel;  
9.     assign tmp_1 = b & (~sel);  
10.    assign q      = tmp_0 | tmp_1;  
11.  
12. endmodule
```

[SystemVerilog] Source Code: mux_2_1_1bit.sv

Notes

- Multiple continuous assignments within a single module.
- All continuous assignment statements are evaluated **concurrently**, the order is of no interest.



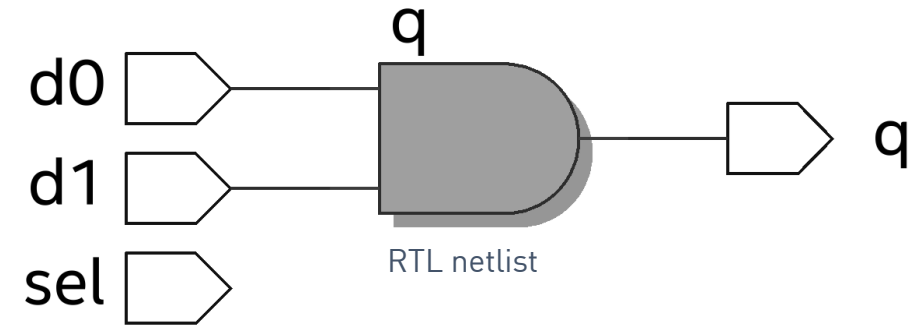
Describing Combinational Logic using SystemVerilog

```
1. module and_2_1bit(  
2.     input logic d0, d1,  
3.     output logic q  
4. );  
5.  
6.     // 2-input AND gate  
7.     always_comb  
8.         q = d0 & d1;  
9.  
10. endmodule
```

1/1

[SystemVerilog] Source Code: and_2_1bit.sv

- Notes
- An alternative way to model combinational logic is to use the **procedural always_comb** construct.



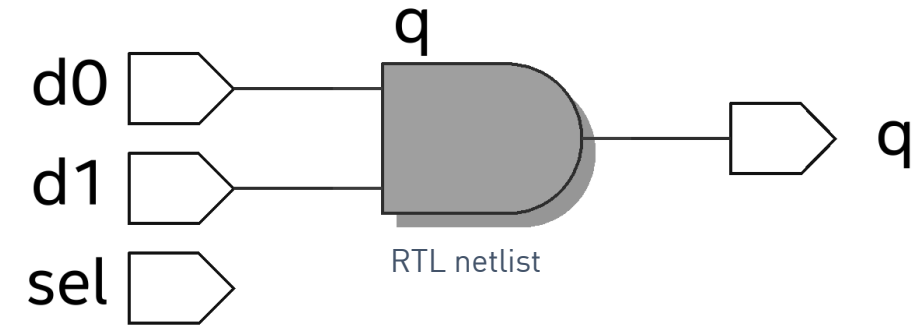
Describing Combinational Logic using SystemVerilog

```
1. module and_2_1bit(  
2.     input logic d0, d1,  
3.     output logic q  
4. );  
5.  
6. // 2-input AND gate  
7. always_comb  
8.     q = d0 & d1;  
9.     ↑ blocking assignment operator  
10. endmodule
```

1/1

[SystemVerilog] Source Code: and_2_1bit.sv

procedural block



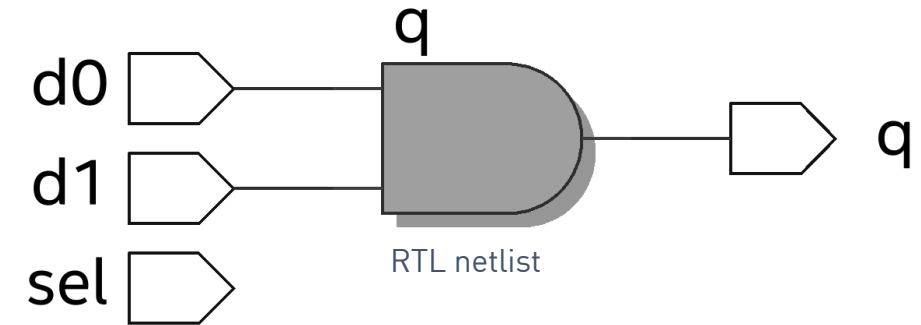
Notes

- **Procedural block:** An alternative way to model combinational logic is to use the procedural `always_comb` construct.
- The meaning of the blocking assignment operator will be discussed on a later stage of this lecture. It comes into play when multiple statements are grouped within a single `always_comb` block.

Describing Combinational Logic using SystemVerilog

```
1. module and_2_1bit(  
2.     input logic d0, d1,  
3.     output logic q  
4. );  
5.  
6.     // 2-input AND gate  
7.     always_comb ← procedural block in  
8.         q = d0 & d1;      SystemVerilog with implicit  
9.                             sensitivity list.  
10. endmodule
```

1/1



[SystemVerilog] Source Code: and_2_1bit.sv

Notes

- The `always_comb` implicitly creates a **complete sensitivity** list including all variables and nets that are read in the process (pretty much the same always as `@*` construct in Verilog-2001).

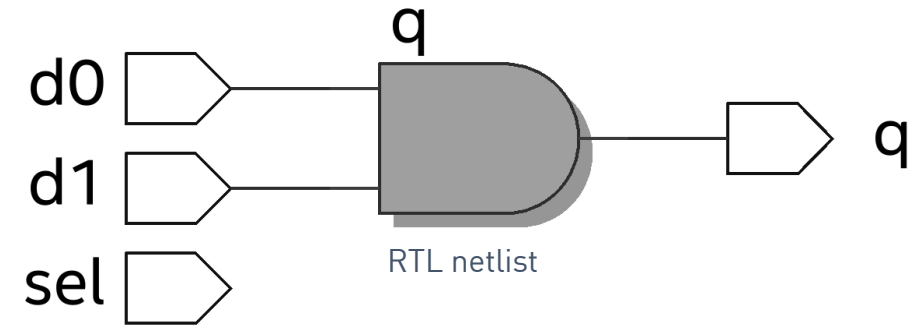
Describing Combinational Logic using SystemVerilog

```
1. module and_2_1bit(  
2.     input wire d0, d1,  
3.     output wire q  
4. );  
5.  
6.     // 2-input AND gate  
7.     always@(d0 or d1) ←  
8.         q = d0 & d1;  
9.  
10. endmodule
```

1/1

Verilog !

procedural block in **Verilog**
with **explicit** sensitivity list.



[SystemVerilog] Source Code: and_2_1bit.v

Notes

- **Sensitivity list:** If an input signal that is specified in the sensitivity list changes its state, all related outputs are re-evaluated.
- The Verilog-2001 standard introduced the use of commas “,” to separate items in the sensitivity list.

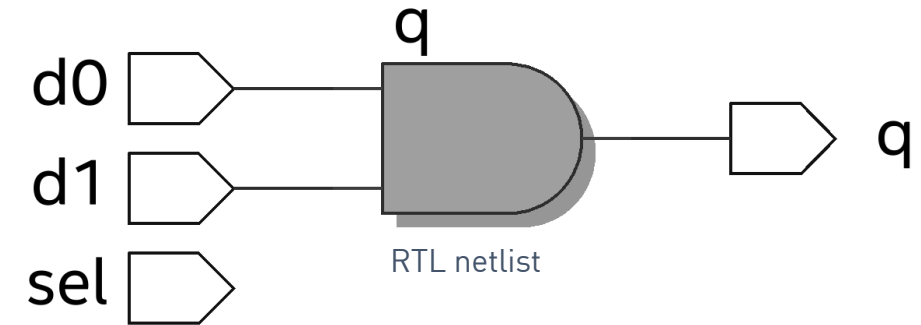
Describing Combinational Logic using SystemVerilog

```
1. module and_2_1bit(  
2.     input wire d0, d1,  
3.     output wire q  
4. );  
5.  
6. // 2-input AND gate  
7. always@*  
8.     q = d0 & d1;  
9.  
10. endmodule
```

1/1

Verilog !

← procedural block in Verilog
with **implicit** sensitivity list.



[SystemVerilog] Source Code: and_2_1bit.v

Notes

- Missing items in the sensitivity list has been a critical issue in Verilog and often led to undesirable and erroneous behavior.
- Therefore, the Verilog 2001 standard a wildcard construct (* - asterisk) that is pretty much the same as the always_comb statement in SystemVerilog.

Describing Combinational Logic using SystemVerilog

```
1. // ...
2. always_comb begin
3.     q_0 = d_0;
4.     q_1 = d_0 & d_1;
5.     q_2 = ~d_1;
6.     ...
7.
8. end
```

1/1

we use the **begin/end** keywords to group multiple statements

sequentially executed

blocking assignment operator

Inside the **always_comb** block, we describe the behaviour of combinational logic in a sequential, algorithmic way with **if**, **else**, **while** and **case** statements.

[SystemVerilog] Source Code Snippet

Notes

- We use blocking assignments within an procedural `always_comb` block to model the behavior of combinational logic gates.
- Blocking assignments evaluate the RHS of an expression and update the LHS immediately before any other instruction is executed.
- The order of statements within a procedural `always_comb` block matters!

Describing Combinational Logic using SystemVerilog

```
1. // ...  
2. always_comb begin  
3.     ...  
4. end  
5.   
6. always_comb begin  
7.     ...  
8. end  
9.
```

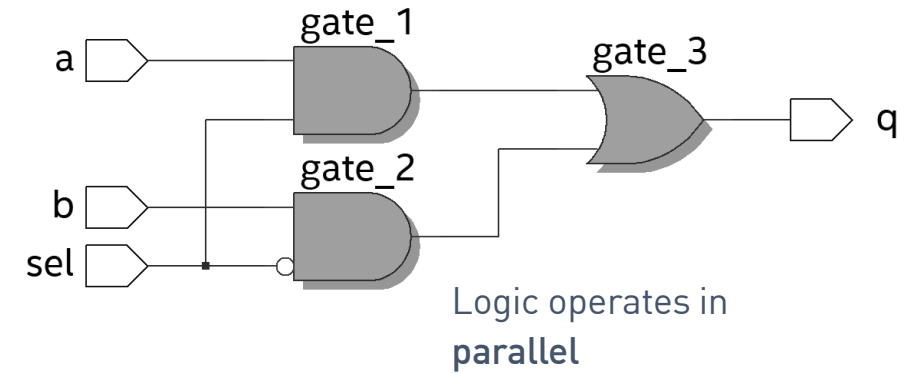
1/1

← Parallel, concurrent execution

[SystemVerilog] Source Code Snippet

Notes

- **Logic gates operate in parallel:** The number of always blocks in a single module is not limited and all blocks are **executed concurrently**, without any predefined order ...
- All statements inside the begin/end section of a procedural always_comb statement **executed sequentially**.



Describing Combinational Logic using SystemVerilog

```
1.    // ...
2.    always_comb begin
3.        a = ...
4.    end
5.
6.    always_comb begin
7.        a = ...
8.    end
9.
```

1/1

← Multiple-driver error: Both blocks are driving the exact same signal ...

[SystemVerilog] Source Code Snippet

Notes

- Logic signals can only have a single driving source.
- Therefore, a variables on the LHS of a procedural block **cannot be assigned** within another procedural blocks.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

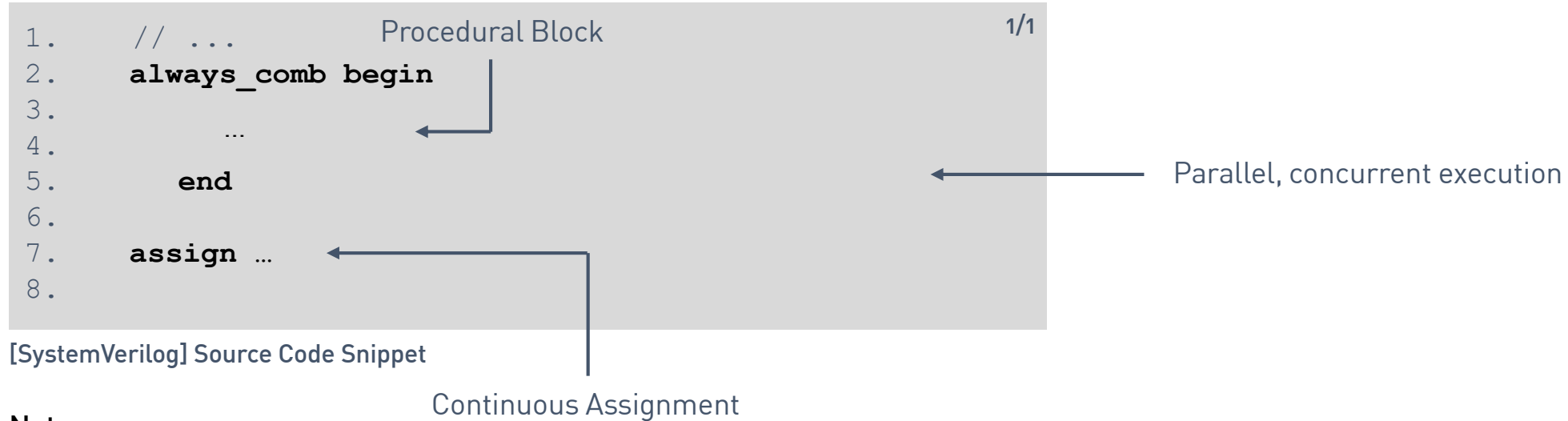
Describing Combinational Logic using SystemVerilog

```
1. // ... Procedural Block 1/1
2. always_comb begin
3.     ...
4.
5. end
6.
7. assign ...
8.
```

Parallel, concurrent execution

Continuous Assignment

[SystemVerilog] Source Code Snippet



Notes

- SystemVerilog models concurrency with two basic constructs: Continuous assignments and procedural blocks.
- Both constructs can co-exist within a single module.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

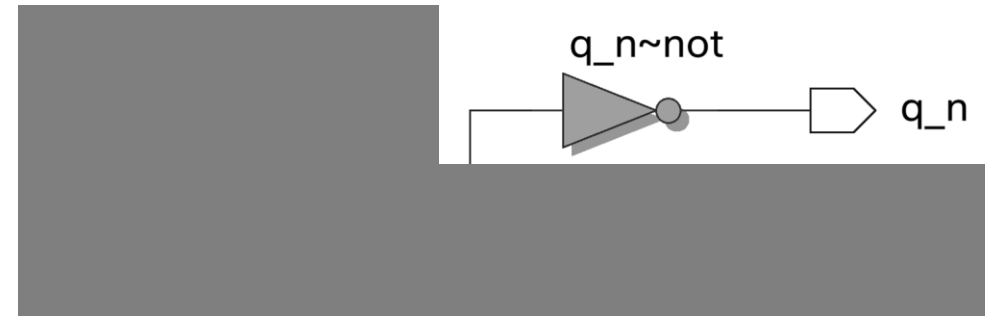
Describing Combinational Logic using SystemVerilog

```
1. module mux_2_1_1bit_n(  
2.     input logic a, b, sel,  
3.     output logic q, q_n  
4. );  
5.   
6.   
7.   
8.   
9.   
10.   
11.   
12.     assign q_n = ~q;   
13.   
14. endmodule
```

Procedural Block

Continuous Assignment

[SystemVerilog] Source Code: mux_2_1_1bit.sv



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Conclusion so far ...

Introduction to SystemVerilog

Conclusion so far ...

- There are two different ways to model combinational logic in SystemVerilog:
 - Procedural blocks: `always_comb` statements
 - Continuous assignments: `assign` statements
- Both constructs can co-exists in a single module.
- They are so called concurrent statements which means that they are executed in parallel and might operate independently of each other.
- In case of continuous assignments, the RHS expression is continuously evaluated as a function of arbitrarily-changing inputs. The target of a continuous assignment is always a net driven by combinational logic.
- A procedural `always_comb` block using blocking assignments allows to model the behaviour of combinational logic in a sequential, algorithmic way using constructs such as `if`, `else`, `while` and `case` statements.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

More about Describing Combinational Logic using SystemVerilog ...

Introduction to SystemVerilog

Describing Combinational Logic using SystemVerilog

- All of the following examples can be either implemented using **procedural blocks** or **continuous assignments**.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Bitwise Operators in SystemVerilog

Introduction to SystemVerilog

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Bitwise Operators in SystemVerilog

```
1. module xor_2_1bit(  
2.     input logic d0, d1,  
3.     output logic q  
4. );  
5.  
6.     // 2-input XOR gate  
7.     assign q = d0 ^ d1;  
8.  
9. endmodule
```

1/1

[SystemVerilog] Source Code: xor_2_1bit.sv

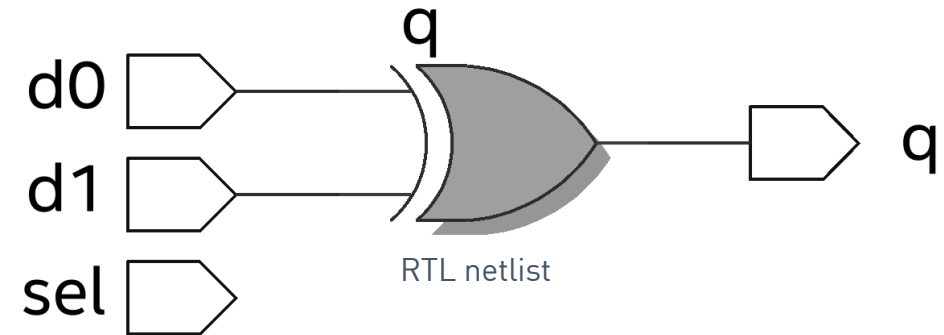
Notes

- Bit-wise AND: &
- Bit-wise OR: |
- Bit-wise XOR: ^
- Bit-wise NOT: ~
- Bit-wise NAND: ~&
- Bit-wise NOR: ~|
- Bit-wise XNOR: ~^

fbeit

FACULTY OF ELECTRICAL ENGINEERING
AND INFORMATION TECHNOLOGY

Prof. Dr. C. Jakob



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Bitwise Operators in SystemVerilog

```
1. module and_2_4bit(  
2.     input logic [3:0] d0, d1,  
3.     output logic [3:0] q  
4. );  
5.       
6.     // four 2-input AND gates  
7.     assign q = d0 & d1;  
8.  
9. endmodule
```

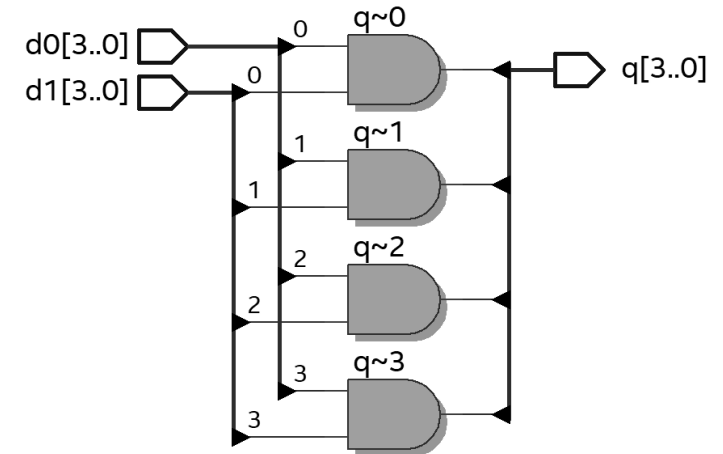
Four bit wide vector, Little-Endian convention

1/1

[SystemVerilog] Source Code: and_2_4bit.sv

Notes

- Bitwise operators operate on either **scalars** (single bit inputs) or **vectors** (multiple bit inputs).
- **Little-Endian**: The value of bit '0' is stored at the vector location with index '0' location



RTL netlist

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Bitwise Operators in SystemVerilog

```
1. module and_2_4bit(
2.     input logic [3:0] d0, d1,
3.     output logic [3:0] q
4. );
5. // four 2-input AND gates
6. always_comb
7.     q = d0 & d1;
8.
9. endmodule
```

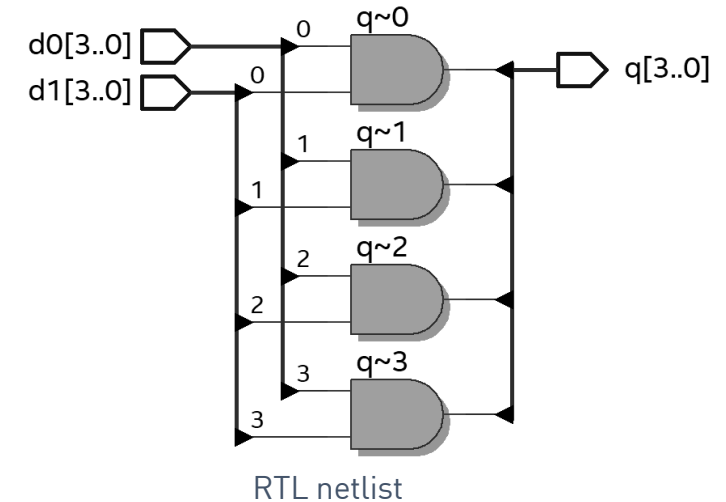
1/1

bit-by-bit operation
on two inputs

[SystemVerilog] Source Code: and_2_4bit.sv

Notes

- Bitwise operators operate on either **scalars** (single bit inputs) or **vectors** (multiple bit inputs).



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Bitwise Operators in SystemVerilog

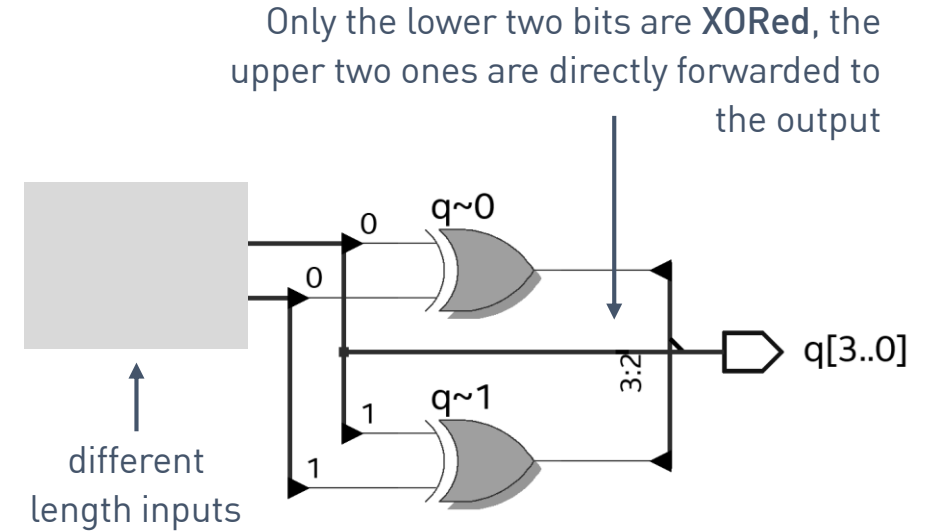
```
1. module or_2_4bit(  
2.     input logic [3:0] d0, input logic [1:0] d1,  
3.     output logic [3:0] q  
4. );  
5.  
6. // four 2-input XOR gates  
7. assign q = d0 ^ d1;  
8.  
9. endmodule
```

1/1

[SystemVerilog] Source Code: or_2_4bit.sv

Notes

- It is not strictly necessary that both inputs have the same length: If one input is not as long as the other, it will automatically be left-extended with zeros to match the length of the other input.



Introduction to SystemVerilog

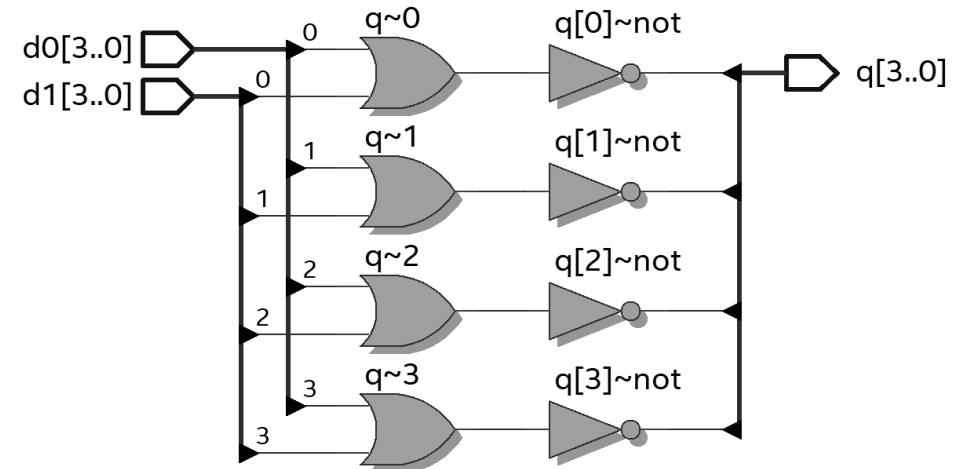
h_da – fbeit - FPGA-based SoC Design

Bitwise Operators in SystemVerilog

```
1. module nor_2_4bit(  
2.     input logic [3:0] d0, d1,  
3.     output logic [3:0] q  
4. );  
5.  
6.     // four 2-input NOR gates  
7.     assign q = ~(d0 | d1);  
8.  
9. endmodule
```

1/1

[SystemVerilog] Source Code: nor_2_4bit.sv



RTL netlist

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Relational Operators in SystemVerilog

Introduction to SystemVerilog

Relational Operators in SystemVerilog

```
1. // greater than ...
2. q = d0 > d1;
3.
4. // greather than or equal to ...
5. q = d0 >= d1;
6.
7. // less than ...
8. q = d0 < d1;
9.
10. // less than or equal to ...
11. q = d0 <= d1;
12.
13. // equal to ...
14. q = d0 == d1;
15.
16. // not equal to
17. q = d0 != d1;
```

1/1

Relational operators evaluate to a 1-bit value, representing true and false respectively.

Relational operators are used to compare the value of two different variables in SystemVerilog.

[SystemVerilog] Source Code Snippet

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Logical Operators in SystemVerilog

Introduction to SystemVerilog

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Logical Operators in SystemVerilog

```
1. module logical_and_2_4bit(
2.     input logic [3:0] d0, d1,
3.     output logic q
4. );
5.
6.     assign q = (d0 && d1);
7.
8. endmodule
```

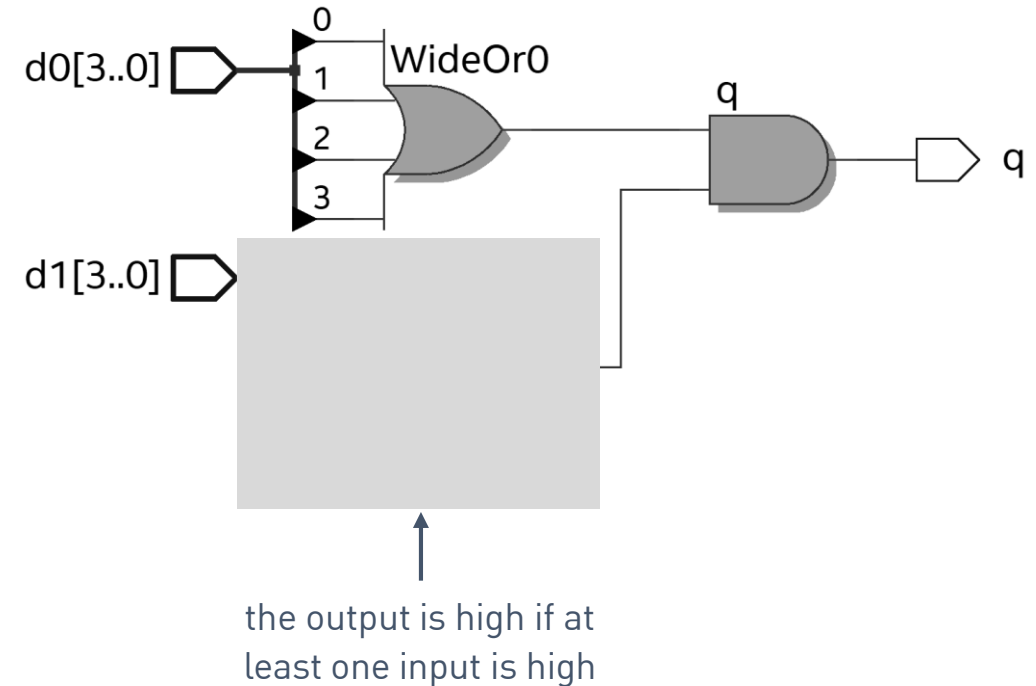
1/1

[SystemVerilog] Source Code: logical_and_2_4bit.sv

Notes

- Logical operators evaluate to a 1-bit value.
- All operands not equal to zero are equivalent to one.
- SystemVerilog Logical Operators:

Logical AND:	&&
Logical OR:	
Logical NOT:	!



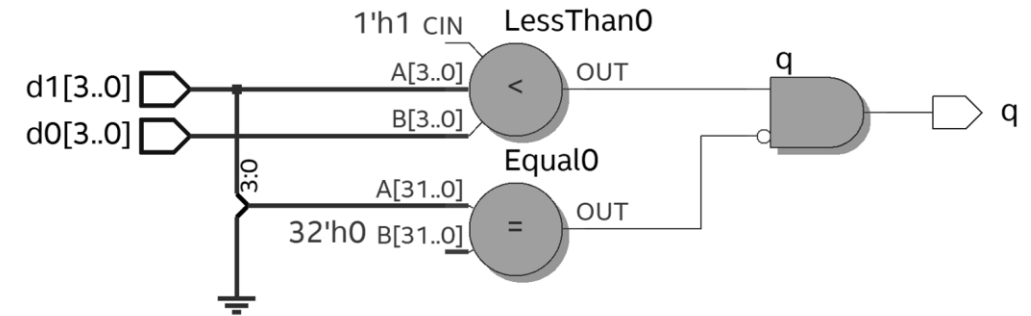
Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Logical Operators in SystemVerilog

```
1. module sys_proc(  
2.     input logic [3:0] d0, d1,  
3.     output logic q  
4. );  
5.  
6.     assign q = (d0 >= d1) && (d1 != 0);  
7.  
8. endmodule
```

1/1



[SystemVerilog] Source Code: sys_proc.sv

Notes

- Rather than using these operators to model gates we use them to **combine relational operators**.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

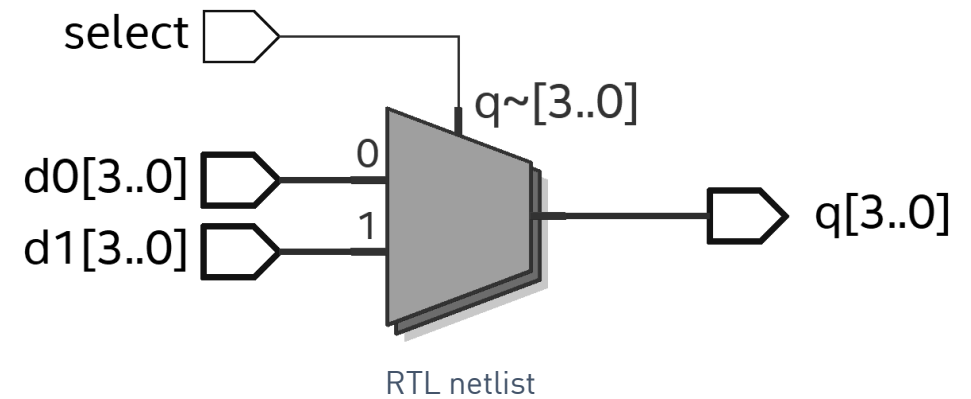
Multiplexers in SystemVerilog

Introduction to SystemVerilog

Multiplexers - Conditional assignment

```
1. module mux_2_4bit(  
2.     input logic [3:0] d0, d1, input logic select,  
3.     output logic [3:0] q  
4. );  
5.  
6.     assign q = (select == 1) ? d1 : d0;  
7.  
8. endmodule
```

1/1



[SystemVerilog] Source Code: mux_2_4bit.sv

Notes

- The ? expression is also called a **ternary operator** because it operates on three inputs: `select`, `d0` and `d1`.
- For more complex structures including several sequentially placed multiplexers, use the `always_comb` construct in conjunction with if-else or case statements ...

Multiplexers - Conditional assignment

```
1. module mux_2_4bit(  
2.     input logic [3:0] d0, d1, input logic select,  
3.     output logic [3:0] q  
4. );  
5.  
6.     assign q = select ? d1 : d0;  
7.  
8. endmodule
```

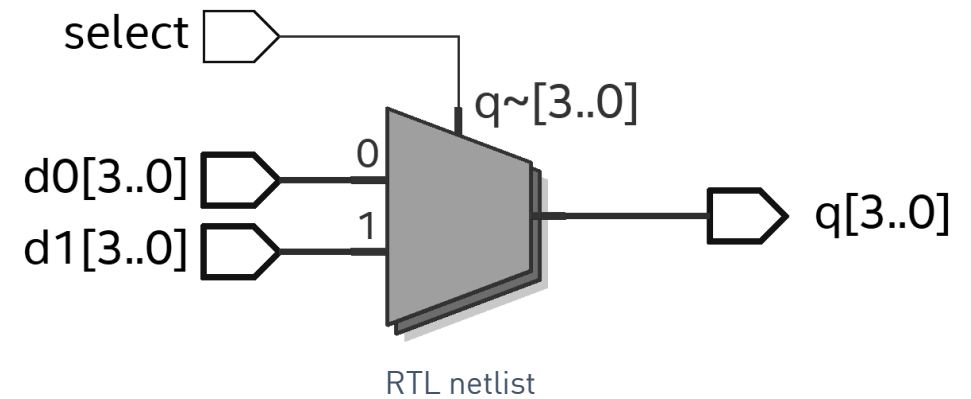
1/1

[SystemVerilog] Source Code: mux_2_4bit.sv

Notes

- The same functionality as before ...

A slightly more compact way of specifying the select signal. This is possible if select is a single bit signal ...



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

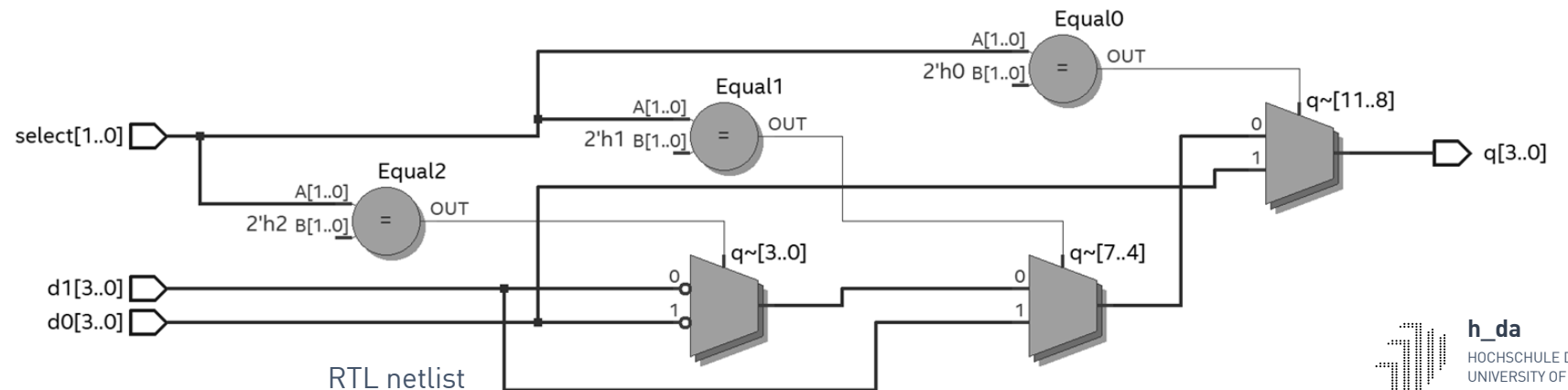
Multiplexers - Conditional assignment

Two input mux with 2-bit select signal ...

```
1. module multistage_mux_2_4bit(  
2.     input logic [3:0] d0, d1, input logic [1:0] select,  
3.     output logic [3:0] q  
4. );  
5.  
6.     assign q = (select == 2'b00) ? d0 : (select == 2'b01) ? d1 : (select == 2'b10) ? ~d0 : ~d1;  
7.  
8. endmodule
```

1/1

[SystemVerilog] Source Code: multistage_mux_2_4bit.sv



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Multiplexers - Conditional assignment

```
1. module multistage_mux_2_4bit(  
2.     input logic [3:0] d0, d1, input logic [1:0] select,  
3.     output logic [3:0] q  
4. );  
5.  
6.     assign q = (select == 2'b00) ? d0 :  
7.               (select == 2'b01) ? d1 :  
8.               (select == 2'b10) ? ~d0 : ~d1;  
9. endmodule
```

1/1

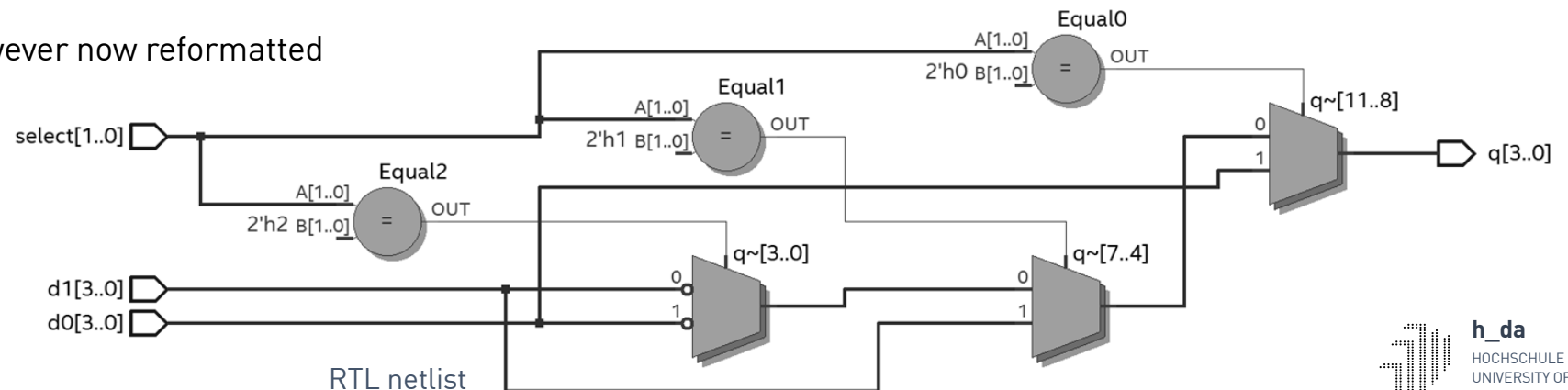
Conditions

[SystemVerilog] Source Code: multistage_mux_2_4bit.sv

Respective output selections

Notes

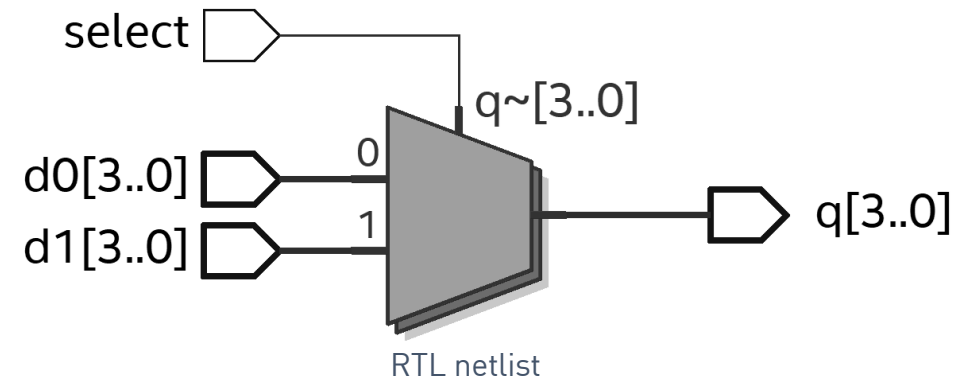
- The same module as before, however now reformatted to improve the readability.



Multiplexers – always_comb

```
1. module mux_2_4bit_comb(  
2.     input logic [3:0] d0, d1, input logic select,  
3.     output logic [3:0] q  
4. );  
5.     always_comb  
6.         if(select == 1'b1)  
7.             q = d1;  
8.         else  
9.             q = d0;  
10. endmodule
```

1/1



[SystemVerilog] Source Code: mux_2_4bit_comb.sv

Notes

- The same as before, but now using a procedural `always_comb` block in conjunction with an if-else statement.

Multiplexers – always_comb

```
1. module multistage_mux_2_4bit_comb(
2.     input logic [3:0] d0, d1, input logic [1:0] select,
3.     output logic [3:0] q
4. );
5.
6.     always_comb
7.         if(select == 2'b00)
8.             q = d0;
9.         else if(select == 2'b01)
10.            q = d1;
11.        else if(select == 2'b10)
12.            q = ~d0;
13.        else
14.            q = ~d1;
15. endmodule
```

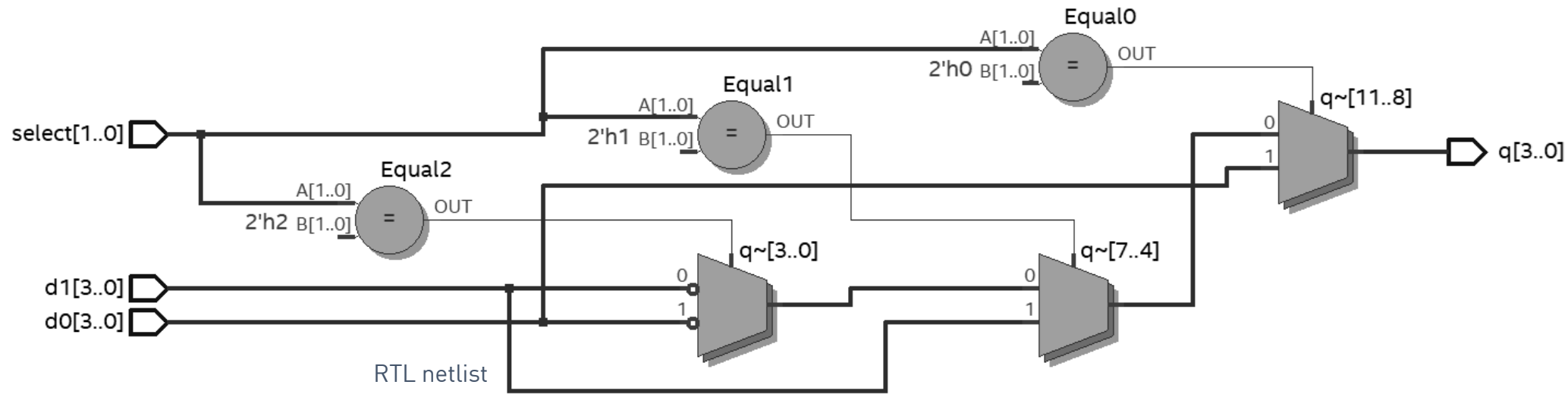
1/1

[SystemVerilog] Source Code: multistage_mux_2_4bit_comb.sv

Notes

- The same as before (multistage_mux_2_4bit.sv), but now with a if/else-if statement within the always_comb block.

Multiplexers – always_comb



Notes

- The synthesized netlist is the same as before ...

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Comparators in SystemVerilog

Introduction to SystemVerilog

Comparators - Conditional assignment

```
1. module comp_gt_4bit(  
2.     input logic [3:0] d0, d1,  
3.     output logic q  
4. );  
5.  
6.     assign q = (d0 > d1) ? 1'b1 : 1'b0;  
7.  
8. endmodule
```

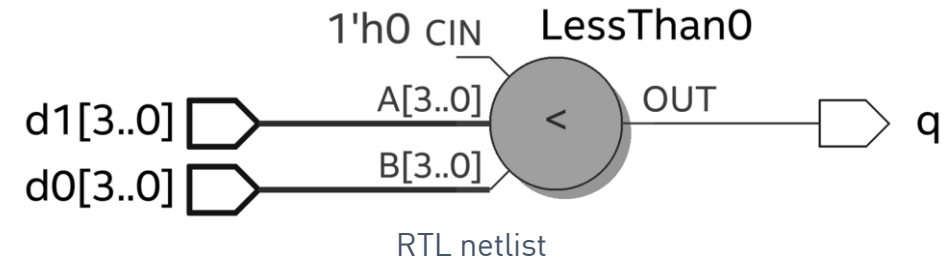
1/1

[SystemVerilog] Source Code: comp_gt_4bit.sv

Notes

- Output q is set to one if the input vector d0 is larger than d1
- All other relational operators are possible as well:

a greater than b	a > b
a greater than or equal to	a >= b
a less than b	a < b
a less than or equal to b	a <= b



Here, we use a conditional assignment to implement a greater-than comparator

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Nested if-else statements might lead to priority logic ...

Introduction to SystemVerilog

Encoder – nested if statements

```
1. module encoder_6_4_bit(  
2.     input logic [3:0] d0, d1, d2, d3, d4, d5,  
3.     input logic [2:0] select, output logic [3:0] q  
4. );  
5.     always_comb  
6.         if(select == 3'd0)  
7.             q = d0;  
8.         else if(select == 3'd1)  
9.             q = d1;  
10.        else if(select == 3'd2)  
11.            q = d2;  
12.        else if(select == 3'd3)  
13.            q = d3;  
14.        else if(select == 3'd4)  
15.            q = d4;  
16.        else  
17.            q = d5;  
18. endmodule
```

1/1

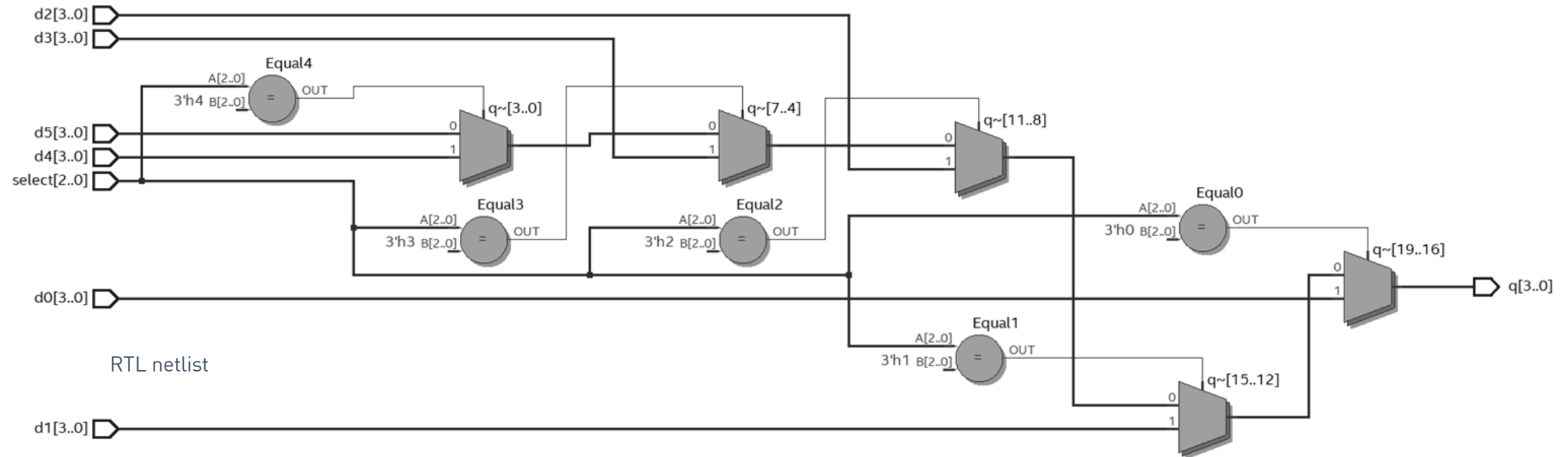
Notes

- Consider the example of an encoder implemented using a set of nested if-statements.
- The task of the encoder is to compress multiple binary inputs into a smaller number of outputs.
- In the present example, we've got a 6-to-1 selection.
- The drawback of using nested if-statements becomes visible by observing the synthesis netlist generated.



[SystemVerilog] Source Code: encoder_6_4_bit.sv

Encoder – nested if statements



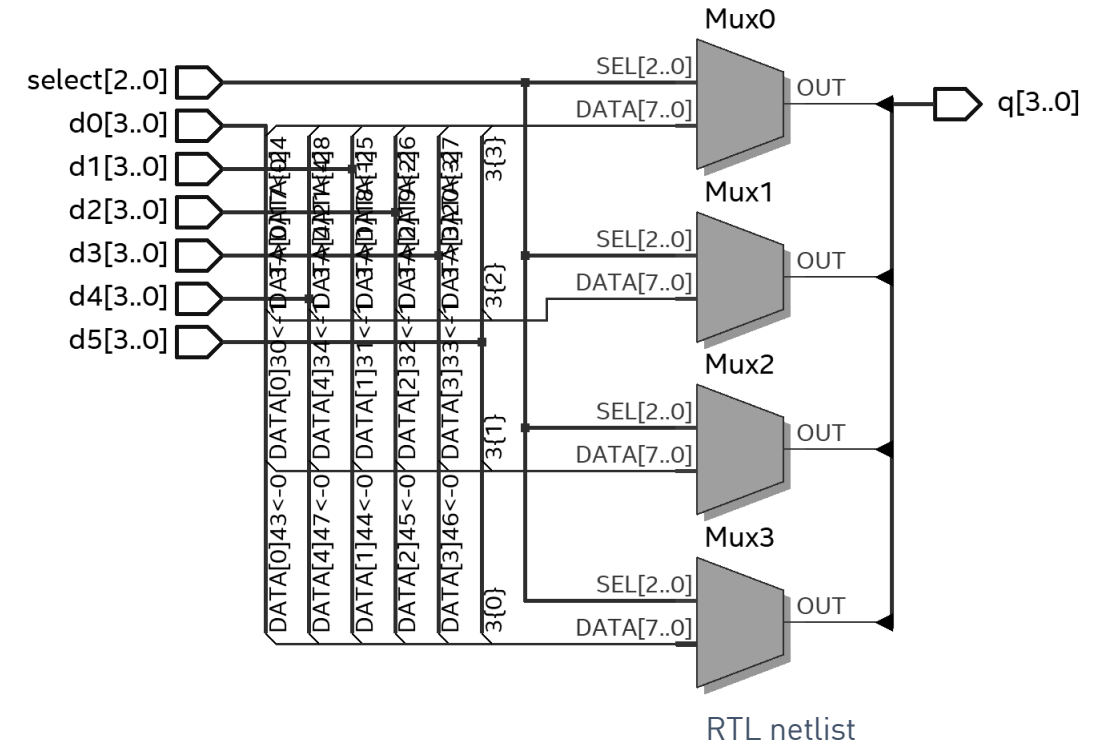
Notes

- Nested if-statements might lead to priority logic, which adds both extra logic gates and longer timing paths to the logic.
- The cascaded logic affects the performance of the circuit due to an increase of area and delay. Observe the path of input `d5` to the output `q`: It passes five logic stages (muxes) while input `d0` only traverses a single one ...

Encoder – case statements

```
1. module encoder_6_4_bit_case(  
2.     input logic [3:0] d0, d1, d2, d3, d4, d5,  
3.     input logic [2:0] select, output logic [3:0] q  
4. );  
5.     always_comb  
6.         case(select)  
7.             0: q = d0;  
8.             1: q = d1;  
9.             2: q = d2;  
10.            3: q = d3;  
11.            4: q = d4;  
12.            default:  
13.                q = d5;  
14.        endcase  
15. endmodule
```

1/1



[SystemVerilog] Source Code: encoder_5_4_bit_case.sv

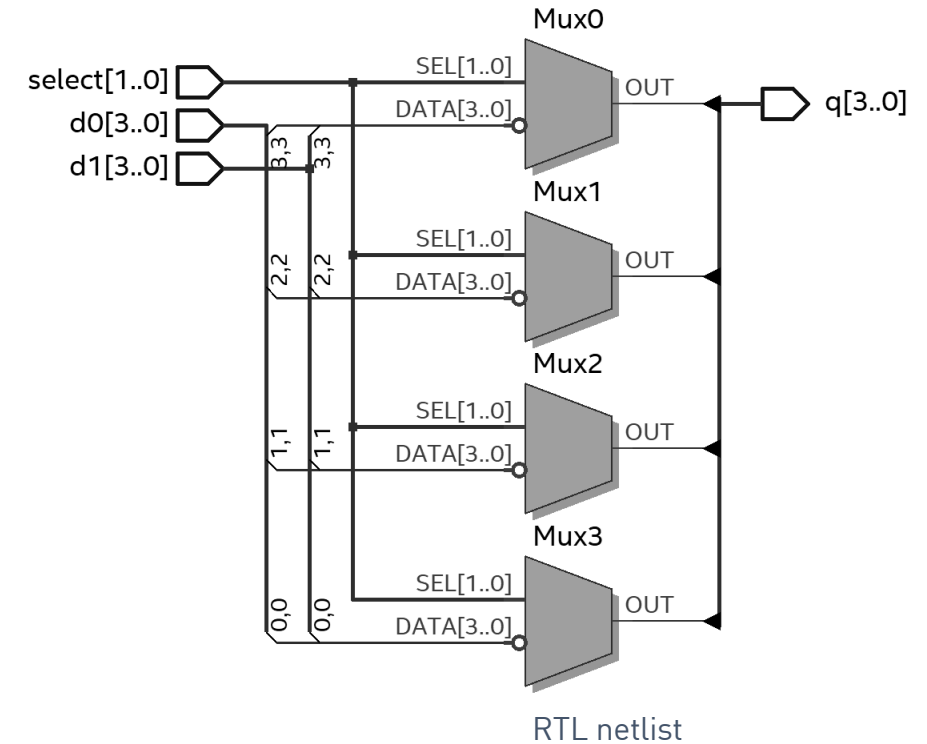
Notes

- If the use-case or application doesn't require an explicit priority scheme, it is possible to replace the nested if-statements by a case statement based implementation. All cases are now matched in parallel.

Multiplexers – always_comb

```
1. module multistage_mux_2_4bit_comb(
2.     input logic [3:0] d0, d1, input logic [1:0] select,
3.     output logic [3:0] q
4. );
5.
6.     always_comb
7.         case(select)
8.             0: q = d0;
9.             1: q = d1;
10.            2: q = ~d0;
11.            3: q = ~d1;
12.        endcase
13.
14. endmodule
```

1/1



[SystemVerilog] Source Code: multistage_mux_2_4bit_comb.sv

Notes

- The same as before, but now using an case statement instead of nested if-statements. The synthesized netlist is different right now, the functionality is however still the same as before ...

Selecting arithmetic operations

```
1. module multistage_mux_4_4bit_comb(
2.     input logic [3:0] d0, d1, input logic [2:0] select,
3.     output logic [3:0] q_0, q_1
4. );
5. always_comb
6.     case(select)
7.         0: begin q_0 = d0; q_1 = d1; end ← simple branch
8.         1: begin
9.             q_0 = d0 + d1;
10.            q_1 = d0 - d1; ← group multiple statements with begin/end
11.        end
12.        2,3,4 : begin q_0 = ~d1; q_1 = ~d0; end ← match multiple select values
13.        default: begin
14.            q_0 = 4'b0000;
15.            q_1 = 4'b0000;
16.        end
17.    endcase
18. endmodule
```

↑ default outputs for all other select values

[SystemVerilog] Source Code: multistage_mux_4_4bit_comb.sv

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Select arithmetic operations

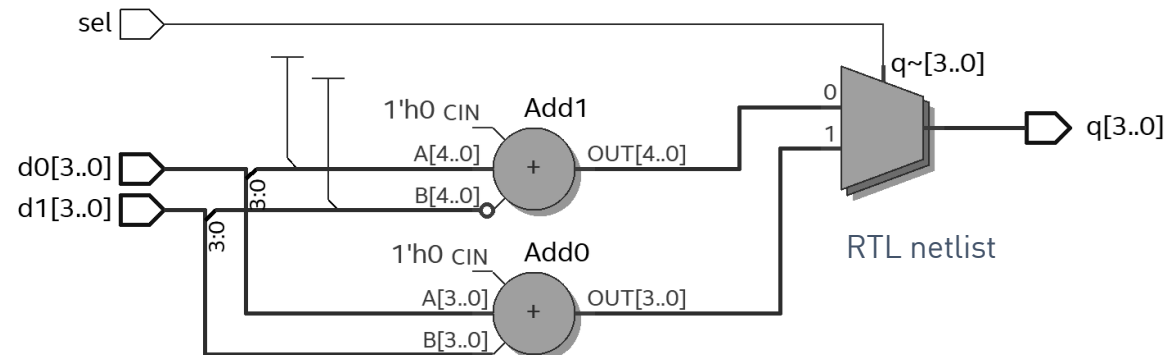
```
1. module op2_sel(  
2.     input logic [3:0] d0, d1, input logic sel,  
3.     output logic [3:0] q  
4. );  
5.  
6.     always_comb  
7.         if(sel)  
8.             q = d0 + d1;  
9.         else  
10.            q = d0 - d1;  
11.  
12. endmodule
```

1/1

Notes

- Both operations are computed in parallel.
- A multiplexer is used to select the sum or the difference computed.

[SystemVerilog] Source Code: op2_sel.sv



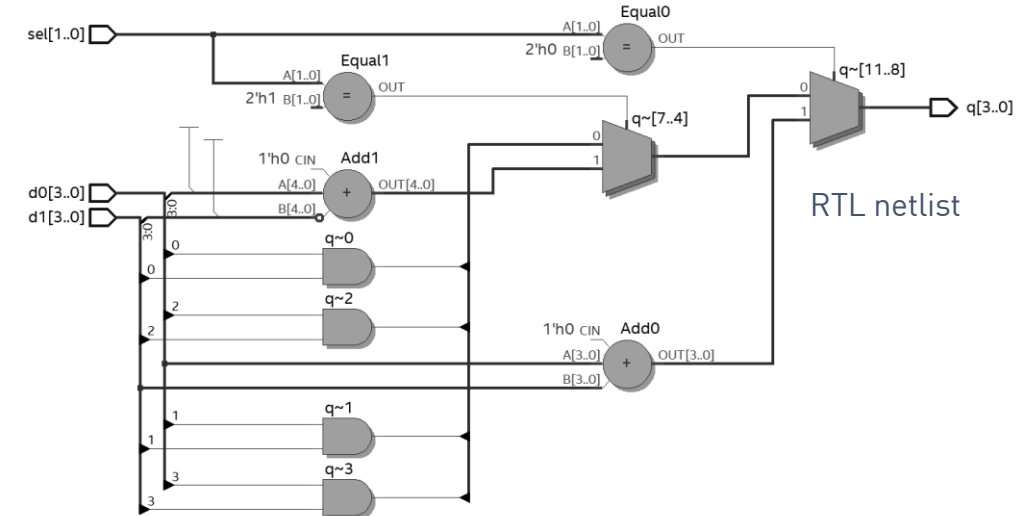
Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Select arithmetic operations

```
1. module op3_sel(  
2.     input logic [3:0] d0, d1, input logic [1:0] sel,  
3.     output logic [3:0] q  
4. );  
5.  
6.     always_comb  
7.         if(sel == 2'b00)  
8.             q = d0 + d1;  
9.         else if(sel == 2'b01)  
10.            q = d0 - d1;  
11.        else  
12.            q = d0 & d1;  
13.  
14. endmodule
```

1/1



[SystemVerilog] Source Code: `op3_sel.sv`

Notes

- Again, all three operations are computed in parallel. Two (cascaded) multiplexers implement priority, i.e. the all zero state for `sel` is prioritized over non-zero states.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Select arithmetic operations

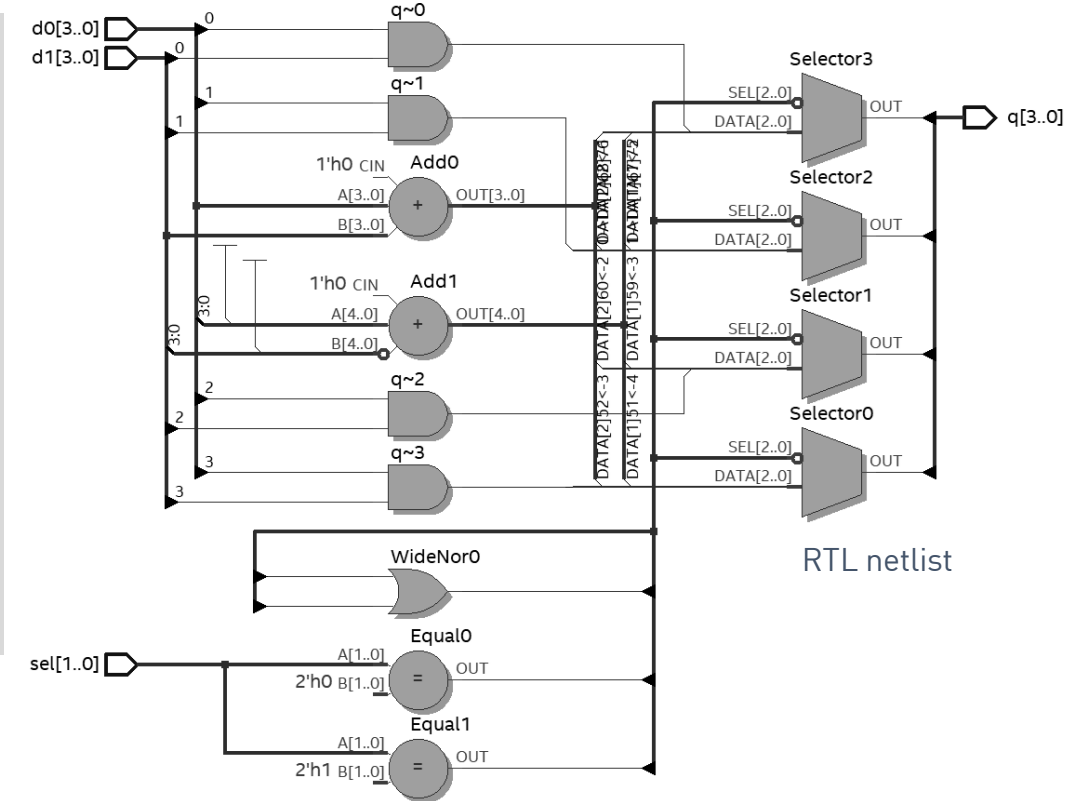
```
1. module op3_sel_case(  
2.     input logic [3:0] d0, d1, input logic [1:0] sel,  
3.     output logic [3:0] q  
4. );  
5.  
6.     always_comb  
7.         case(sel)  
8.             0: q = d0 + d1;  
9.             1: q = d0 - d1;  
10.            default:  
11.                q = d0 & d1;  
12.        endcase  
13. endmodule
```

1/1

[SystemVerilog] Source Code: op3_sel_case.sv

Notes

- All operations are computed in parallel without any priority logic implemented.





Appendix

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

History of SystemVerilog

- Verilog was developed by Gateway Design Automation as a proprietary language for logic simulation (Verilog-XL simulator) in 1984. The term Verilog is made-up of the two words "verification" and "logic".
- Gateway Design Automation was acquired by Cadence Design Systems in 1989.
- Verilog became an open standard in 1990 when Cadence Design Systems released Verilog to the public domain. The administration of the language was assigned to the Open Verilog International (OVI) organization which was founded in the same year. OVI had the task of taking the language through the IEEE standardization process.
- In 1995, Verilog was adopted as an IEEE standard (IEEE standard 1364-1995, also denoted as Verilog-95).
- A significantly revised version of Verilog-95 became an IEEE standard in 2001 (IEEE standard 1364-2001, also known as Verilog-2001).

History of SystemVerilog contd.

- The Accellera Systems Initiative was founded in 2000 through the unification of Open Verilog International and VHDL International. Accellera was established as an independent, non-profit organization dedicated to create, support, promote, and advance system-level design, modeling, and verification standards.
- In the early 2000s, Accellera started to develop SystemVerilog as an extension of the Verilog IEEE 1364-2001 standard.
- In 2005, SystemVerilog was adopted as an IEEE standard (1800-2005).
- In the same year, a (slightly) modified and extended Version of Verilog-2001 became an IEEE Standard (IEEE 1364-2005).
- In 2009, the SystemVerilog standard (IEEE 1800-2005) was merged with the base Verilog (IEEE 1364-2005) standard, forming IEEE Standard 1800-2009.
- The current version is IEEE standard 1800-2012.